

DEEP LEARNING & GenAI

CONSOLIDATED EXAM STUDY NOTES

IIT Bombay | ePGD AI & DS | May 2026

Open Book Exam · 3 Hours · 50 Marks

Topics Covered:

A: N-grams & Smoothing · B: RNN/LSTM/GRU · C: Seq2Seq & Attention
D: Transformer & Self-Attention · E: BERT / GPT / T5 / BART
F: Prompting & Instruction Tuning · G: RLHF & Alignment
H: Agentic AI · I: Tokenization (BPE/WordPiece/Unigram) · J: Advanced Prompting · K: Code Walkthroughs

WHAT'S INSIDE THIS CONSOLIDATED FILE:

- Part 1: Study Notes — Topics A to K (beginner-friendly, all formulas explained)
- Part 2: Deep-Dive — Transformer Self-Attention (full derivations & examples)
- Part 3: Deep-Dive — RLHF & Alignment (complete pipeline, all objectives)
- Part 4: Formula Dictionary — every symbol explained in plain English
- Part 5: Fully Worked Solutions — step-by-step numerical answers
- Part 6: Architecture Comparison Tables & One-Page Cheat Sheets

Duplicate content removed. Notes 1 & 2 are subsets of the Complete Notes and are not repeated.

PART 1 — STUDY NOTES: TOPICS A to K

Source: DL_EndSem_Complete_Notes.docx — the most comprehensive version. Notes from DL_EndSem_Study_Notes.docx and DL_EndSem_Study_Notes_Part2.docx are fully contained here and are NOT repeated.

DEEP LEARNING & GenAI

COMPREHENSIVE END-SEM EXAM STUDY NOTES

IIT Bombay | ePGD AI & DS | May 2026

Exam: 3 Hours | 50 Marks | 11 Topics | All Lectures Covered

A: N-grams & Smoothing	B: RNN / LSTM / GRU	C: Seq2Seq & Attention
D: Self-Attention & Transformer	E: BERT / GPT / T5 / BART	F: Prompting & IFT
G: RLHF & Alignment	H: Agentic AI Frameworks	I: Tokenization (BPE/WP/ULM)
J: Advanced Prompting & POSIX	K: Code Walkthroughs (4 Labs)	+ Formula Sheet Exam Q&A

TABLE OF CONTENTS

TOPIC A: N-GRAM LANGUAGE MODELS, SMOOTHING & PERPLEXITY

1. Intuition & Key Concepts

A language model assigns probability to a sequence of words. N-gram models do this using counts from a corpus, based on the Markov assumption.

1.1 N-gram Model

- Unigram: $P(w)$ — probability of word w alone
- Bigram: $P(w_2 | w_1) = \text{count}(w_1 w_2) / \text{count}(w_1)$
- Trigram: $P(w_3 | w_1 w_2) = \text{count}(w_1 w_2 w_3) / \text{count}(w_1 w_2)$
- General N-gram: $P(w_i | w_1 \dots w_{i-1}) \approx P(w_i | w_{i-2} \dots w_{i-1})$ [Markov assumption]

$$P(w_1 w_2 \dots w_i) = P(w_1) \times P(w_2 | w_1) \times P(w_3 | w_1 w_2) \times \dots \text{ [chain rule]}$$

► **EXAM TIP:** the bigram matrix is NOT symmetric. But the sum of row i and sum of column i for any word i ARE the same (marginal counts).

1.2 Smoothing Techniques

Problem: Zero probability for unseen n-grams. Solution: Smoothing.

Add-1 (Laplace) Smoothing:

$$P(w_i | w_{i-1}) = [\text{count}(w_{i-1}, w_i) + 1] / [\text{count}(w_{i-1}) + |V|]$$

- $|V|$ = vocabulary size. Adds 1 to all counts.
- Problem: Steals too much probability from high-count bigrams. Like 'Robin Hood' — takes from the rich, gives to the poor.

Good-Turing Smoothing:

- Key idea: Use count of things seen ONCE to estimate count of things NEVER seen.
- N^c = frequency of frequency c (how many n-grams appear exactly c times)

$$c^* = (c+1) \times N(c+1) / N(c) \text{ [modified count for Good-Turing]}$$

- For unseen ($c=0$): $P = N_1 / N$ (N_1 = number of singletons, N = total tokens)

Kneser-Ney (KN) Smoothing [MOST IMPORTANT]:

- Instead of unigram probability, uses CONTINUATION PROBABILITY

$$P_{\text{continuation}}(w) = |\{v : \text{count}(v, w) > 0\}| / \sum_{w'} |\{v : \text{count}(v, w') > 0\}|$$

- Intuition: 'Angeles' has high unigram count but only follows 'Los'. Coffee follows many words → higher continuation probability.

$$P_{\text{KN}}(w_i | w_{i-1}) = \max(\text{count}(w_{i-1}, w_i) - D, 0) / \text{count}(w_{i-1}) + \lambda(w_{i-1}) \cdot P_{\text{continuation}}(w_i)$$

- $D \approx 0.75$ (absolute discount, empirically derived). λ ensures probabilities sum to 1.

1.3 Perplexity (Evaluation Metric)

Intrinsic evaluation: no downstream task needed. Intuition: average branching factor (how many words could come next).

$$PP(W) = P(w_1 w_2 \dots w_N)^{-1/N} = 2^{H(W)}$$

- $H(W) = -1/N \sum \log_2 P(w_i | \text{context})$ [cross-entropy per word]
- Lower perplexity = better model (more certain about next word)
- $PP=100$ means model is as confused as choosing randomly from 100 words

► **INFO:** $\text{Perplexity} = 2^{\text{entropy}}$. Connection: $\text{entropy} = \text{average bits to encode}$; $\text{perplexity} = \text{number of equally likely choices}$.

1.4 Problems with N-gram Models

Problem	Impact	Solution
Sparsity (zero counts)	Many n-grams never seen	Smoothing
Fixed window size	Can't capture long dependencies	
RNN/Transformer		
Exponential data need	Larger n = exponentially more data	Neural LMs
No generalization	Can't handle OOV words well	Word embeddings

2. Sample Questions & Solutions

[EASY] What is perplexity and what does a lower value mean?

ANS: $\text{Perplexity} = P(\text{test_set})^{-1/N}$. Measures how well a language model predicts a test set. Lower perplexity = better model = more certain about the next word. $PP=8$ means the model is as confused as choosing randomly among 8 words.

[MEDIUM] A corpus has: I love pizza (×3), I love pasta (×2), I hate pizza (×2), I hate cats (×1). Compute bigram $P(\text{pizza} | \text{love})$ with Add-1 smoothing ($|V|=6$).

ANS: $\text{count}(\text{love}, \text{pizza})=3$, $\text{count}(\text{love})=3+2=5$. Add-1: $P = (3+1)/(5+6) = 4/11 \approx 0.364$. Without smoothing: $3/5=0.6$. The smoothed value is lower because probability mass is redistributed.

[HARD] Why does KN smoothing use continuation probability instead of unigram probability? Give an example where unigram probability fails.

ANS: Unigram counts are dominated by frequent words like 'Angeles' (high count due to 'Los Angeles'). But 'Angeles' only completes one unique bigram (with 'Los'). KN's continuation count = number of unique left contexts = 1 for 'Angeles' vs ~10 for 'coffee'. Thus $P_{\text{KN}}(\text{coffee}) > P_{\text{KN}}(\text{Angeles})$ as a continuation word, which is linguistically correct.

3. Quick Revision — Topic A

► **INFO:** Bigram: $P(w|v)=c(v,w)/c(v)$. Add-1: +1 numerator, +|V| denominator. Good-Turing: $c^*=(c+1)Nc+1/Nc$. KN: continuation probability, best smoothing. Perplexity= $P(W)^{-1/N}=2^H$; lower=better.

TOPIC B: RNN, LSTM, GRU — SEQUENCE MODELS

1. Vanilla RNN

An RNN processes sequences by maintaining a hidden state that carries information from previous time steps.

```
h_t = tanh(W_h h_{t-1} + W_e x_t + b)
y_t = softmax(U · h_t)
```

- W_h : hidden-to-hidden weight matrix (shared across all time steps)
- W_e : input embedding weight matrix (shared across all time steps)
- Key advantage: shared weights — model size does NOT grow with sequence length

1.1 Training: Backpropagation Through Time (BPTT)

```
∂L/∂W = ∑_t ∑_{s=0}^T (∂h_t/∂h_s) (∂h_s/∂W)
```

- Chain of Jacobians: each $\partial h_t / \partial h_s = \prod (\partial h_i / \partial h_{i-1})$
- If $|J| < 1$: Vanishing gradient → model forgets distant context
- If $|J| > 1$: Exploding gradient → training diverges

NOTE: Exploding gradient: fixed by gradient clipping. Vanishing gradient: fixed by LSTM/GRU, residual connections, or truncated BPTT.

1.2 Key Disadvantages of RNN

- Sequential processing: cannot parallelize (must compute h_t before h_{t+1})
- Vanishing gradient: forgets long-range dependencies
- Teacher forcing: use ground truth (not model output) as next input during training

2. LSTM (Long Short-Term Memory)

LSTM adds a CELL STATE (C_t) as explicit memory alongside the hidden state (h_t). Three gates control information flow.

2.1 LSTM Gates

Gate	Formula	Function
Forget Gate (f_t)	$\sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$ (0=forget all, 1=keep all)	How much of C_{t-1} to keep
Input Gate (i_t)	$\sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$	How much new content to write to cell
Cell Content ($\tilde{C}_t$)	$\tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$	New candidate content for cell state
Cell Update (C_t)	$f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$	Updated memory cell
Output Gate (o_t)	$\sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$	How much cell to expose as hidden state
Hidden State (h_t)	$o_t \odot \tanh(C_t)$	Output / next input to next layer
$C_t = f_t \odot C_{t-1} + i_t \odot \tanh(W_c \cdot [h_{t-1}, x_t])$ [key cell update equation]		
$h_t = o_t \odot \tanh(C_t)$		

► **EXAM TIP:** If $f_t=1$ and $i_t=0$: cell state perfectly copied — no vanishing gradient! LSTM reduces (not eliminates) vanishing gradient.

► **EXAM TIP:** LSTM does NOT guarantee solving vanishing gradient — it REDUCES the problem.

3. GRU (Gated Recurrent Unit)

GRU simplifies LSTM: no separate cell state, only 2 gates instead of 3.

```
z_t = σ(Wz · [h_{t-1}, x_t]) [update gate]
r_t = σ(Wr · [h_{t-1}, x_t]) [reset gate]
h_tilde = tanh(W · [r_t ⊙ h_{t-1}, x_t]) [candidate hidden state]
h_t = (1 - z_t) ⊙ h_{t-1} + z_t ⊙ h_tilde [final hidden state]
```

3.1 LSTM vs GRU Comparison

Aspect	RNN	LSTM	GRU
States		h_t only	$h_t + C_t$ (cell) h_t only
Gates	None	3 (f, i, o)	2 (z, r)
Parameters		Fewest	Most (4×) Medium (3×)
Handles long deps	Poor	Best	Good
Training speed	Fastest	Slowest	Medium
Rule of thumb	—	Start here	Switch if data is small

4. Bidirectional RNN & Multi-Layer RNN

- BiRNN: Two RNNs — one left-to-right, one right-to-left. Final state = $[h \rightarrow ; h \leftarrow]$. Used for tasks where full sequence is available (POS tagging, NER).
 - Multi-layer RNN: Output of layer 1 hidden states becomes input to layer 2. More capacity.
- **EXAM TIP:** Use BiRNN for classification tasks (sequence available). Use uni-directional for generation (future unknown).

5. Sample Questions & Solutions

[EASY] What problem does LSTM solve compared to vanilla RNN, and how?

ANS: LSTM solves the vanishing gradient problem. It adds a cell state C_t with additive (not multiplicative) updates: $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$. When $f_t=1$, $i_t=0$, gradients flow through the cell state unchanged (no multiplication by small Jacobians), preserving information from distant time steps.

[MEDIUM] What is 'teacher forcing' and why is it used? What's its main drawback?

ANS: Teacher forcing: during training, feed the GROUND TRUTH previous token as input (not the model's prediction). Benefit: faster, more stable training. Drawback: exposure bias — at inference time, model sees its own (possibly wrong) outputs, which can cause error accumulation.

[HARD] If $f_t=0$ and $i_t=1$ in LSTM at all time steps, what happens to the cell state? If $f_t=1$ and $i_t=0$?

ANS: Case 1 ($f_t=0$, $i_t=1$): $C_t = 0 + \tilde{C}_t = \tilde{C}_t$. Cell completely overwrites — no memory of past. Case 2 ($f_t=1$, $i_t=0$): $C_t = C_{t-1}$. Cell perfectly copied — infinite memory (gradient highway). This is why LSTM can preserve information for long sequences.

6. Quick Revision — Topic B

► **INFO:** RNN: $h_t = \tanh(W_{hh}h_{t-1} + W_{ex}x_t)$. Vanishing gradient → LSTM adds cell state C_t with forget/input/output gates. GRU: 2 gates (update+reset), no separate cell. Bidirectional: use when full sequence available. Teacher forcing: feed ground truth not predictions during training.

TOPIC C: SEQ2SEQ MODELS & ATTENTION MECHANISM

1. Seq2Seq Architecture

Two RNNs: Encoder processes source sentence → Decoder generates target sentence. Used for: machine translation, summarization, dialogue.

$$P(y_1 \dots y_{\square} | x_1 \dots x_{\square}) = \prod P(y_{\square} | y_1 \dots y_{\square-1}, h_{\square_encoder}) \quad [\text{conditional LM}]$$

- Encoder: reads full input, produces final hidden state $h_{\square}$
 - Decoder: generates output tokens one-by-one, conditioned on $h_{\square}$
- **EXAM TIP:** BOTTLENECK PROBLEM: Single vector $h_{\square}$ must encode entire input. Long sequences → encoder forgets early words. Solution: Attention!

1.1 Beam Search

Greedy decoding picks highest probability token at each step — can't backtrack. Beam search keeps top-k (beam width) hypotheses at each step.

$$\text{Score}(y_1 \dots y_{\square}) = \sum \log P(y_i | y_1 \dots y_{i-1}, x) / t \quad [\text{length-normalized}]$$

- Larger beam width = HIGHER probability sequences (NOT lower — exam question!)
 - Normalize by length to avoid preference for short sequences
- **EXAM TIP:** 'As beam size increases, decoder outputs sequences of SMALLER probability' → FALSE. Larger beam = higher probability sequences.

2. Attention Mechanism

Gives decoder DIRECT ACCESS to all encoder hidden states. Removes bottleneck.

2.1 Attention Computation

$e_{\square, \square} = s_{\square} \cdot h_{\square}$ [attention score: dot product of decoder state $s_{\square}$ with encoder state $h_{\square}$]

$\alpha_{\square, \square} = \text{softmax}(e_{\square, \square})$ [attention distribution]

$a_{\square} = \sum \alpha_{\square, \square} \cdot v_{\square}$ [attention vector = weighted sum of VALUES] ← EXAM CRITICAL

► **EXAM TIP:** the output vector $a_{\square} = \sum \alpha_{\square, \square} \cdot v_{\square}$ (weighted sum of VALUES, NOT queries or keys!)

2.2 Attention Variants

Type	Formula	Notes
------	---------	-------

Dot Product	$q \cdot k$	Basic, fast
-------------	-------------	-------------

Scaled Dot Product	$q \cdot k / \sqrt{d_k}$	Used in Transformer — prevents saturation
--------------------	--------------------------	---

Bilinear	$q^T W k$	Learned similarity
----------	-----------	--------------------

Additive (Bahdanau)	$v^T \tanh(W_1 q + W_2 k)$	Original attention paper
---------------------	----------------------------	--------------------------

2.3 RAG — Retrieval Augmented Generation

Model	Key Property	Single-doc output?
-------	--------------	--------------------

REALM	Retrieves from corpus, single retrieval step	YES — one doc only
-------	--	--------------------

FiD (Fusion-in-Decoder)	Encodes docs separately, fuses in decoder	NO — combines multiple
-------------------------	---	------------------------

RAG-Token	Different doc per token generation	NO — multiple docs
-----------	------------------------------------	--------------------

RAG-Sequence — one doc per seq	Same doc for full sequence generation	YES
--------------------------------	---------------------------------------	-----

► **EXAM TIP:** Final output from single document → REALM and RAG-Sequence. RAG reduces hallucination vs closed-book LLM.

3. Sample Questions & Solutions

[EASY] What is the attention vector and how is it computed?

ANS: Attention vector $a_t = \sum_i \alpha_{t,i} \cdot v_i$ (weighted average of encoder states/values). Weights $\alpha_{t,i}$ come from softmax of dot products between decoder state s_t and each encoder state h_i . It gives decoder direct access to relevant encoder positions.

[MEDIUM] Why does Bahdanau attention not guarantee one-to-one alignment?

ANS: The attention distribution $\alpha_{t,i}$ is a soft distribution (softmax output). Multiple decoder positions can have high weight for the same encoder position.

[HARD] A seq2seq model translates English → French. Input has 8 tokens; output has 10. How many attention scores are computed total?

ANS: At each decoder step, we compute 8 attention scores (one per encoder state). With 10 decoder steps → $10 \times 8 = 80$ attention scores total. Each score $e_{t,i} = s_t \cdot h_i$ measures relevance of encoder position t for decoder output at position i . After softmax: 10 attention distributions each summing to 1.

TOPIC D: SELF-ATTENTION & TRANSFORMER ARCHITECTURE

1. From Attention to Self-Attention

Key insight: If attention gives decoder access to all encoder states, why keep recurrent connections at all? Remove them → SELF-ATTENTION: every position attends to every other position in the same sequence.

- Enables PARALLEL processing of all tokens (GPU-friendly!)
- BUT: loses position information → need Positional Encoding
- BUT: linear operation → need Feed-Forward Networks for nonlinearity

2. Scaled Dot-Product Attention

$Q = H \cdot W_q$, $K = H \cdot W_k$, $V = H \cdot W_v$ [linear projections of hidden states]
 $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$

- Q (Query): what am I looking for?
- K (Key): what do I contain? (matched against queries)
- V (Value): what do I return when selected?
- $\sqrt{d_k}$ scaling prevents dot products from becoming too large → softmax saturation → vanishing gradients

NOTE: Complexity of self-attention: $O(n^2 \cdot d)$ — quadratic in sequence length n . This is the main scaling bottleneck.

3. Multi-Head Attention

$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W_o$
 $\text{head}_i = \text{Attention}(Q \cdot W_{q_i}, K \cdot W_{k_i}, V \cdot W_{v_i})$

- Each head learns different aspects: syntactic, semantic, positional relationships
- Standard: 8 heads (Transformer-base), 16 heads (Transformer-large)

► **EXAM TIP:** Single head focuses on ONE relationship due to softmax. Multi-head learns DIVERSE relationships in different subspaces. Output = Concat(all heads) then linear projection.

4. Masked Self-Attention (Decoder)

Decoder is autoregressive: at position l , cannot attend to positions $> l$ (future tokens not yet generated).

```
eLt =  $-\infty$  for all  $t > l$  [set future attention scores to  $-\infty$  before softmax]
```

- After softmax: $\exp(-\infty) = 0 \rightarrow$ attention weights for future positions = 0

► **EXAM TIP:** asking preserves auto-regressive property. Applied BEFORE softmax by adding $-\infty$ to upper triangle of attention score matrix.

5. Positional Encoding

Self-attention is permutation-invariant ('bag of words'). Positional encoding injects position information.

```
PE(pos, 2i) =  $\sin(\text{pos} / 10000^{(2i/d_{\text{model}})})$   
PE(pos, 2i+1) =  $\cos(\text{pos} / 10000^{(2i/d_{\text{model}})})$   
Input to transformer = token_embedding + positional_encoding [element-wise addition]
```

► **EXAM TIP:** DECODER-ONLY (GPT) can work without positional encoding. Why? Decoder uses masked attention (causal) — position is implicitly encoded via the masking structure itself.

6. FFN, Layer Norm & Residual Connections

```
FFN(x) =  $\max(0, x \cdot W_1 + b_1) \cdot W_2 + b_2$  [ReLU activation]  
LayerNorm(x) =  $\gamma \cdot (x - \mu) / \sigma + \beta$  [learnable scale  $\gamma$  and shift  $\beta$ ]  
Output = LayerNorm(x + Sublayer(x)) [Add & Norm block]
```

7. Complete Transformer Architecture

Encoder block ($\times N$ layers): [Multi-Head Self-Attention $\rightarrow$ Add&Norm $\rightarrow$ FFN $\rightarrow$ Add&Norm]

Decoder block ($\times N$ layers): [Masked MH Self-Attention $\rightarrow$ Add&Norm $\rightarrow$ Cross-Attention $\rightarrow$ Add&Norm $\rightarrow$ FFN $\rightarrow$ Add&Norm]

Cross-Attention: Q from decoder, K and V from encoder. Links decoder to encoder.

Type	Examples	Training Objective	Best For
Encoder-only	BERT, RoBERTa, ALBERT	Masked Language Modeling (MLM)	Classification, NER, QA

Decoder-only	GPT-2, GPT-3, LLaMA	Causal/Autoregressive LM
Text generation, chat		
Encoder-Decoder	T5, BART, mT5	Autoregressive (encoder+decoder)
Translation, summarization		

8. Sample Questions & Solutions

[EASY] Why do we divide by $\sqrt{d}$ in scaled dot-product attention?

ANS: Without scaling, as d grows, dot products QK^T become very large, pushing softmax into saturation regions where gradients ≈ 0 . Dividing by $\sqrt{d}$ keeps the variance of dot products ≈ 1 , leading to better gradient flow.

[MEDIUM] Explain how cross-attention connects encoder and decoder in Transformer.

ANS: Q comes from DECODER states (after masked self-attention), K and V come from ENCODER final states. Each decoder position queries which encoder positions are most relevant via $Q \cdot K^T$. This is unmasked (decoder queries, encoder has full info). Parameter matrices W_q used with decoder; W_k, W_v used with encoder.

[HARD] Attention complexity is $O(n^2d)$. For $n=512$ and $d=512$, how many multiplications? What's the main architectural solution?

ANS: $O(n^2d) = 512^2 \times 512 = 134M$ multiply-adds per layer. For 12 layers: $\sim 1.6B$ operations. Solutions: (1) Sparse/local attention (Longformer, BigBird). (2) Linear attention (Performer). (3) State space models (Mamba) — $O(n \log n)$. (4) Flash Attention — I/O-aware exact attention.

9. Quick Revision — Topic D

► **INFO:** Self-attention: Q, K, V from same sequence. Scaled: $QK^T/\sqrt{d_k} \cdot V$. Multi-head: H parallel heads, concat+project. Masking: $-\infty$ to future. PE: $\sin/\cos$. Residual+LayerNorm. Cross-attention: Q =decoder, K, V =encoder.

TOPIC E: PRE-TRAINING STRATEGIES — BERT, GPT, T5, BART

1. Word Embeddings → Contextual Representations

Method	Type	Context?	Key Feature
--------	------	----------	-------------

Word2Vec (CBOW/Skip-gram)	Static	No	Predict context or center word
------------------------------	--------	----	--------------------------------

GloVe	Static	No	Global co-occurrence matrix; NO negative sampling
-------	--------	----	---

ELMo	Dynamic	Yes (BiLSTM)	BiLSTM; word embedding depends on context
------	---------	--------------	---

BERT NSP	Dynamic	Yes (Transformer)	Bidirectional Transformer; MLM +
-------------	---------	-------------------	----------------------------------

GPT	Dynamic	Yes (causal)	Unidirectional Transformer; autoregressive
-----	---------	--------------	--

► **EXAM TIP:** 'GloVe uses more negative samples than Word2Vec' → FALSE. GloVe uses NO negative sampling (it uses global co-occurrence matrix).

2. BERT (Bidirectional Encoder Representations from Transformers)

- Encoder-only Transformer: 12 layers (BERT-base) or 24 layers (BERT-large)
- Input = [CLS] + Sentence A + [SEP] + Sentence B + [SEP]
- Embeddings = Token Embedding + Segment Embedding + Positional Embedding

Pre-training Task 1: Masked Language Model (MLM)

Objective: Predict masked tokens

- 15% of tokens selected for masking:

- 80%: Replace with [MASK] token
- 10%: Replace with random word (corruption)
- 10%: Keep original (prevents model from ignoring non-masked)

► **EXAM TIP:** Loss computed ONLY at masked positions. [CLS] attends to ALL tokens — aggregate representation for classification.

Pre-training Task 2: Next Sentence Prediction (NSP)

- [CLS] token → binary classifier: IsNext / NotNext
- 50% positive (consecutive sentences), 50% negative (random pairs)

2.1 Fine-tuning BERT

Task	Input Format	Output Head
------	--------------	-------------

Sentiment/Classification embedding → classes	[CLS] + sentence	Linear on [CLS]
QA (SQuAD) start/end span position	[CLS]+question+[SEP]+passage	Two linear heads:
NER/POS token embedding → tag	[CLS] + sentence	Linear on each
NLI entail/contra/neutral	[CLS]+sent1+[SEP]+sent2	Linear on [CLS] →

3. GPT — Decoder-Only

- Decoder-only Transformer: causal (left-to-right) language modeling

Objective: $P(w_{\square} | w_1, \dots, w_{\square-1})$ [next token prediction]

► **EXAM TIP:** Base GPT models (LLaMA-base) fail zero-shot instruction following because pre-training data is NOT in instruction-output format.

4. T5 & BART

Model	Architecture	Pre-training	Key Idea
T5	Enc-Dec	Span masking (sentinel tokens)	All tasks as text-to-text

BART	Enc-Dec	Denoising (noise+reconstruct)	Denoising
autoencoder; great for summarization			

5. Sample Questions & Solutions

[EASY] Why does BERT use a [CLS] token for classification?

ANS: [CLS] is a special token with NO semantic meaning of its own. In self-attention, it attends to ALL other tokens and aggregates information from the entire sequence without being biased toward any particular word's semantics.

[MEDIUM] BERT uses 3 types of embeddings. Name them and explain why each is needed.

ANS: (1) Token Embeddings: represent word identity (from vocabulary). (2) Segment Embeddings: distinguish Sentence A from Sentence B (needed for NSP). (3) Positional Embeddings: encode token position (Transformer self-attention is permutation-invariant). All three are summed before entering the encoder stack.

[HARD] For SQuAD question answering with BERT, how are start and end span positions determined?

ANS: Two linear heads: for each token position i , compute $P_{\text{start}}(i) = \text{softmax}(W_s \cdot h_i)$ and $P_{\text{end}}(i) = \text{softmax}(W_e \cdot h_i)$. During training: minimize cross-entropy for both. During inference: find $i \leq j$ maximizing $P_{\text{start}}(i) \times P_{\text{end}}(j)$. The span $[i, j]$ in the passage is the predicted answer.

6. Quick Revision — Topic E

► **INFO:** GloVe: NO negative sampling. BERT: MLM(15% masked)+NSP, bidirectional, [CLS]+[SEP]. GPT: causal LM, decoder-only. T5: text-to-text. BART: denoising. Base models fail zero-shot because NOT trained on instruction-output format.

TOPIC F: PROMPTING, IN-CONTEXT LEARNING & INSTRUCTION TUNING

1. Prompting Techniques

Technique	Description	Example
Zero-shot	No examples; just instruction	"Translate to French: Hello"

Few-shot	K examples in prompt	"EN→FR: Hi→Bonjour. Translate: Hello"
Chain-of-Thought	Show reasoning steps in examples	"Q: 2+3×4. Let me think: 3×4=12, 12+2=14. A: 14"
Self-Consistency	Multiple CoT paths → majority vote	Generate N reasoning chains, pick most common answer

Prompt Tuning Learn soft prompt vectors (not text) Prepend learned embedding tokens to input

NOTE: Chain-of-thought only emerges in large models (≥100B params). Small models don't benefit.

2. In-Context Learning (ICL)

- Model learns a task from examples IN THE PROMPT — no weight updates
- Performance is SENSITIVE to: example order, example selection, prompt format

NOTE: Prompt sensitivity is a major problem: same examples in different order can change accuracy by 20%+.

3. Instruction Tuning (IFT)

Fine-tune a pre-trained model on (instruction, response) pairs to make it follow natural language commands.

Dataset: {(instruction_i, output_i)} — thousands of diverse task instances

Method	Key Mechanism	Exam Answer
Self-Instruct	Model generates instructions from seed tasks; filters them	Self-generated
Evol-Instruct	Instruction Evolver: adds constraints, deepens, concretizes prompts	Q1.14 Answer: C

Orca Uses GPT-4 explanation traces; teaches reasoning process
System prompts + CoT

► **EXAM TIP:** 'Instruction Evolver that complicates prompts using constraints/deepening/concretizing' → EVOL-INSTRUCT.

4. Sample Questions & Solutions

[EASY] What is the difference between zero-shot and few-shot prompting?

ANS: Zero-shot: only an instruction, no examples. Model must use pre-training knowledge alone. Few-shot: K examples (input→output) provided in the prompt before the actual query. Examples help the model understand the task format and desired output style.

[MEDIUM] Why does chain-of-thought prompting improve performance on math problems?

ANS: CoT forces intermediate reasoning steps to be verbalized, making errors visible. It breaks complex problems into simpler sub-steps, leveraging the model's ability to do arithmetic on simple sub-problems. Without CoT, model tries to produce the answer in a single forward pass.

[HARD] If a prompt yields 70% accuracy, but shuffling the example order drops it to 50%, what does this suggest? How would you make models more robust?

ANS: This 20% drop shows extreme prompt sensitivity — the model is learning surface correlations (recency bias) rather than true task understanding. Solutions: (1) Self-Consistency: aggregate by majority vote across random orderings. (2) Calibration: subtract base probability of answer labels. (3) Instruction tuning to be ordering-invariant.

5. Quick Revision — Topic F

► **INFO:** Zero-shot→no examples. Few-shot→K examples. CoT→reasoning steps. Prompt sensitive to order/format. Self-Instruct: model generates tasks. Evol-Instruct: Instruction Evolver. IFT insufficient: doesn't prevent harmful outputs.

TOPIC G: RLHF — REINFORCEMENT LEARNING FROM HUMAN FEEDBACK

1. RLHF Pipeline Overview

Stage	What Happens	Output
Stage 1: SFT	Supervised fine-tuning on curated instruction-response pairs	SFT model (reference policy)
Stage 2: Reward Model	Train RM on human-ranked responses using Bradley-Terry model	Reward function $r(x,y)$

Stage 3: RL Optimization Use PPO to maximize reward while staying close to SFT model RLHF-tuned policy π_θ

2. Reward Model Training

Bradley-Terry Model:

$$P(y_1 > y_2 \mid x) = \exp(r(x, y_1)) / [\exp(r(x, y_1)) + \exp(r(x, y_2))]$$

Reward Model Loss:

$$L(RM) = -\log \sigma(r\theta(x, y^+) - r\theta(x, y^-)) \quad [\text{maximize } \log \sigma \text{ of reward difference}]$$

► **EXAM TIP:** reward model objective = MAXIMIZE $\log \sigma(r(x, y_1) - r(x, y_2))$. This is equivalent to maximizing log-likelihood under Bradley-Terry.

- RLAIIF: another LLM (e.g., GPT-4) annotates instead of humans
- Constitutional AI: model uses 16 principles to self-annotate

3. RL Formulation for LLMs

RL Concept LLM Equivalent

Policy π_θ	LLM that generates tokens (parameterized by weights θ)
State s_t	Prompt + all tokens generated so far up to time t

Action a_t Single output token generated at time t

Reward r Given only at end of sequence (EOS token), not per-token

4. RLHF Objective Function

$$\max_{\pi_\theta} E_{y \sim \pi_\theta(y|x)} [r(x, y) - \beta \cdot \log(\pi_\theta(y|x) / \pi_{\text{ref}}(y|x))]$$

- First term: maximize expected reward from reward model
- Second term: KL divergence penalty — prevent policy from deviating too far from reference (SFT model)
- KL penalty prevents 'reward hacking': model gaming reward model with verbose/emoji-heavy outputs

► **EXAM TIP:** LHF objective has TWO terms: (1) $E[r(x, y)]$ and (2) $-\beta \cdot \text{KL}(\pi_\theta || \pi_{\text{ref}})$. The KL term is the regularizer preventing reward hacking.

5. Value Functions & PPO

Function Formula Meaning

$Q(s, a)$	$E[\sum \gamma^t \cdot r_{t+k} \mid s_t=s, a_t=a]$	Expected cumulative reward after taking action a in state s
-----------	--	---

$V(s)$	$E[\sum \gamma^t \cdot r_{t+k} \mid s_t=s]$	Expected value of being in state s (averaged over all actions)
--------	---	--

$A(s, a) = Q(s, a) - V(s)$ Advantage of action a over average action in state s

$$L_{\text{PPO}} = E[\min(r_\square \cdot A_\square, \text{clip}(r_\square, 1-\epsilon, 1+\epsilon) \cdot A_\square)]$$

$$r_\square = \pi_\theta(a_\square | s_\square) / \pi_{\theta_{\text{old}}}(a_\square | s_\square) \quad [\text{importance ratio}]$$

► **EXAM TIP:** PPO clips importance ratio to $[1-\epsilon, 1+\epsilon]$ to prevent large policy jumps. Allows reusing old policy samples via importance sampling.

6. Sample Questions & Solutions

[EASY] What is reward hacking in RLHF and how does the KL penalty prevent it?

ANS: Reward hacking: policy learns to maximize reward model score through superficial features (verbose responses, emoticons, repetition). KL penalty $-\beta \cdot \text{KL}(\pi_\theta || \pi_{\text{ref}})$ penalizes the policy for deviating too far from the reference (SFT) model, keeping responses grounded in human-like language.

[MEDIUM] Explain why REINFORCE with sequence-level reward is problematic. How does the advantage function help?

ANS: With sequence reward R , every token gets the same reward. If 'Taj Mahal is in France' gets $R=-1$, even 'Taj' (correct) gets penalized. High variance: 'Taj' could start both good and bad sequences. Advantage $A(s,a)=Q(s,a)-V(s)$ measures RELATIVE quality of token 'a' vs average token at state s , reducing variance.

[HARD] Write the complete RLHF objective and explain each term. Under what condition is the KL term zero?

ANS: Objective: $\max_{\theta} E_{y \sim \pi_{\theta}(y|x)} [r(x,y) - \beta \cdot \log(\pi_{\theta}(y|x) / \pi_{\text{ref}}(y|x))]$.
Term 1: $E[r(x,y)]$ = maximize expected reward. Term 2: $-\beta \cdot \text{KL}(\pi_{\theta} || \pi_{\text{ref}})$ = KL regularizer preventing reward hacking. $\text{KL}=0$ iff $\pi_{\theta} = \pi_{\text{ref}}$ exactly (no policy change). Setting $\beta=\infty$ prevents any update; $\beta=0$ gives pure reward maximization with reward hacking risk.

7. Quick Revision — Topic G

► **INFO:** Pipeline: SFT → Reward Model → PPO. RM loss: $\max \log \sigma(r(x,y^+) - r(x,y^-))$ [Bradley-Terry]. RLHF obj: $\max E[r(x,y) - \beta \cdot \text{KL}(\pi_{\theta} || \pi_{\text{ref}})]$. State=prompt+tokens. Action=token. Reward=EOS. $A(s,a)=Q(s,a)-V(s)$. PPO-clip: ratio $\in [1-\epsilon, 1+\epsilon]$. Constitutional AI: 16 principles, RLAI.

TOPIC H: AGENTIC AI FRAMEWORKS

1. Agentic vs Non-Agentic Workflows

Aspect Non-Agentic (Zero-shot) Agentic

Process Single forward pass → output Iterative: plan → act → observe → refine

Error recovery No backtracking Can revise based on feedback

Limitation Fixed by model capacity Can exceed via tool use and iteration

2. Key Agentic Frameworks

2.1 ReAct (Reason + Act)

- Interleaves: Thought → Action → Observation → Thought → ...
- Action: tool call (e.g., Search[query], Calculator[expr])
- Observation: result returned by tool (external knowledge)

► **EXAM TIP:** eAct's 'Observation' step distinguishes it from CoT — CoT relies on internal knowledge only; ReAct gets EXTERNAL feedback from tools.

2.2 Self-Refine

- Cycle: Initial Output → Feedback → Refine → Feedback → Refine → ...
- SAME LLM plays all roles (generator, critic, refiner) — no additional model needed

► **EXAM TIP:** Self-Refine cycle = Initial Output → Feedback (identify shortcomings) → Refine. Feedback IDENTIFIES problems; Refine FIXES them.

2.3 Reflexion

- Verbal reinforcement learning: no weight updates
- Agent fails → generates verbal reflection → stored in episodic memory → informs next trial

► **EXAM TIP:** Reflexion improves via VERBAL REINFORCEMENT LEARNING (self-reflection in text), NOT weight updates.

2.4 ReWOO (Reasoning WithOut Observation)

- DECOUPLES planning from tool execution
- Step 1 (Planner): generate ALL plans/tool calls upfront
- Step 2 (Worker): execute tools in parallel
- Step 3 (Solver): combine results → final answer

► **EXAM TIP:** 'Decouples reasoning from execution to reduce token consumption' → REWOO.

2.5 HuggingGPT

Stage Function

Task Planning Parse user request → structured list of sub-tasks with dependencies

Model Selection Choose best HuggingFace model for each sub-task

Task Execution Run selected models on sub-tasks

Response Generation Aggregate results → final response

3. Framework Comparison

Framework Key Mechanism Distinguishing Feature

ReAct Think-Act-Observe loop External tool observations (real-world grounding)

Self-Refine Generate-Feedback-Refine Same model as generator+critic; no tools

Reflexion Verbal RL via self-reflection No weight updates; episodic memory buffer

ReWOO Plan-Execute-Solve separated No interleaved thinking; most token-efficient

HuggingGPT LLM orchestrates AI models Task Planning → Model Selection pipeline

4. Sample Questions & Solutions

[EASY] What is the difference between ReAct and standard Chain-of-Thought prompting?

ANS: CoT reasons step-by-step using only internal (parametric) knowledge. ReAct adds an 'Action' step (e.g., Search, Calculator) after each Thought, then receives an 'Observation' (external tool result). This allows solving knowledge-intensive tasks where internal knowledge is stale or insufficient.

[MEDIUM] HuggingGPT receives: 'Generate an image of a cat and describe it in English and French.' How does it decompose this task?

ANS: Stage 1 (Task Planning): decompose into: T1=Generate cat image [image-generation], T2=Describe image in English [image-captioning, depends on T1], T3=Translate to French [translation, depends on T2]. Stage 2 (Model Selection): T1→stable-diffusion, T2→BLIP-2, T3→Helsinki-NLP/opus-mt. Stage 3 (Execution): run in dependency order. Stage 4: combine outputs.

[HARD] Why is the 'Observation' step in ReAct crucial? Give a concrete case where CoT would fail but ReAct would succeed.

ANS: Observation brings EXTERNAL, CURRENT information. Example: 'Who is the current CEO of OpenAI?' CoT: uses parametric knowledge (may be outdated). ReAct: Thought='I need current info' → Action=Search['OpenAI CEO 2025'] → Observation='Sam Altman...' → correct answer. External observation grounds reasoning in facts, not hallucinated parametric recall.

5. Quick Revision — Topic H

► **INFO:** ReAct: Thought-Action-Observation. Self-Refine: Output→Feedback→Refine (same LLM). Reflexion: verbal RL, episodic memory, NO weight updates. ReWOO: Plan ALL tools first, execute parallel. HuggingGPT: Task Planning→Model Selection→Execute→Respond.

TOPIC I: TOKENIZATION STRATEGIES — BPE, WordPiece, Unigram LM

Tokenization is the critical first step for ALL modern NLP systems. It splits raw text into tokens that models can process. The right tokenization strategy determines how well a model handles OOV words, morphology, and low-resource languages.

1. Why Sub-word Tokenization?

Level	Vocab Size	OOV Risk	Problem	Used By
Word-level at test time		100k+	High Old NLP	Cannot handle unseen words
Character-level morphological structure	~100	None	Some specialized	Sequences too long; loses
Sub-word (BPE/WP/ULM) + OOV	30k-50k	Low	All modern LLMs	Balanced: handles morphology

► **INFO:** All three sub-word methods share a 2-step framework: (1) Token Learner — runs at TRAIN time, builds vocab. (2) Token Segmenter — runs at TEST time, splits text using learned vocab/rules.

2. Byte-Pair Encoding (BPE)

► **INFO:** Used by: GPT, GPT-2, RoBERTa, BART. Original use: text compression (1994); adapted for NLP in 2016.

Core idea: Start from individual characters, then iteratively merge the most frequently co-occurring pair into a new token.

2.1 BPE Algorithm — Step by Step

1. Pre-tokenization: Clean corpus, split into words using spaces/punctuation.
2. Build base vocabulary: Extract all unique characters. These form the initial vocabulary.
3. Count all character bigram frequencies in the corpus.
4. Merge the highest-frequency pair → add merged token to vocab. Record merge rule.
5. Replace all occurrences of the old pair with the new token.
6. Repeat steps 3-5 until vocabulary reaches the target size (e.g., 30,000 tokens).

2.2 BPE Example

Words: cat(×10), bat(×5), bag(×12), tag(×4), cats(×5). Base vocab: {a, b, c, g, s, t}

```
Step 1: freq(a,t)=20 is highest → merge: a+t → at. Rule: a,t → at
Step 2: freq(a,g)=16 is highest → merge: a+g → ag. Rule: a,g → ag
Step 3: freq(c,at)=15 is highest → merge: c+at → cat. Rule: c,at → cat
```

2.3 BPE Test Time

- Apply merge rules IN ORDER (same sequence as learned)
- 'bags': b,a,g,s → apply a+g→ag: b,ag,s → final: [b, ag, s]
- Unknown characters → replaced by <UNK>

► **EXAM TIP:** BPE merges based on RAW FREQUENCY. At test time: apply stored merge rules in sequence. Unknown CHARACTER becomes <UNK>.

3. WordPiece Tokenization

► **INFO:** Used by: BERT, mBERT, DistilBERT. Internal tokens prefixed with ## to mark non-starting positions.

Similar to BPE but merges based on LIKELIHOOD (PMI-like score), not raw frequency.

$$\text{Score}(A, B) = \text{freq}(A, B) / (\text{freq}(A) \times \text{freq}(B)) \quad [\text{Pointwise Mutual Information}]$$

This penalizes high-frequency pairs where individual tokens are also very frequent. Rare token pairs that often appear together get higher priority.

3.1 ## Prefix Convention

The word 'token' is split as: t, ##o, ##k, ##e, ##n — the ## marks non-starting (internal) positions.

3.2 WordPiece Test Time (KEY DIFFERENCE FROM BPE)

- WordPiece does NOT store merge rules — only stores the final vocabulary.
- At test time: find the LONGEST subword in the vocabulary that matches the start of the word.
- Split off that subword, then repeat on the remainder.
- If any subword is NOT in vocabulary → the ENTIRE WORD becomes <UNK>

Example: 'fused' with vocab {f, ##u, ##s, ##ed}: → f | ##u | ##s | ##ed

► **EXAM TIP:** WordPiece: if ANY sub-token is missing from vocab, the WHOLE WORD becomes <UNK>. BPE: only the unknown CHARACTER becomes <UNK>. — A CRITICAL DISTINCTION!

4. Unigram Language Model (ULM) Tokenization

► **INFO:** Used by: SentencePiece (T5, XLNet, ALBERT). Unlike BPE/WordPiece, ULM allows MULTIPLE segmentation candidates per word.

Key distinction: BPE and WordPiece GROW the vocabulary (start small, expand). ULM SHRINKS vocabulary (start large, prune).

4.1 ULM Algorithm

7. Start with LARGE base vocabulary (all chars + frequent subwords; or run BPE first).
8. For each word: enumerate ALL possible segmentations. Compute probability using Unigram LM (product of independent token probabilities).

$$P(t_1, t_2, \dots, t_n) = \prod_i P(t_i) \quad [\text{independence assumption}]$$

9. Find 'best split' = highest-probability segmentation (using Viterbi algorithm).

10. Compute total log-likelihood loss: $\text{Loss} = \sum_w \text{freq}(w) \times -\log P(\text{best_segmentation}(w))$

11. Remove the vocabulary token whose removal LEAST increases the loss (5-20% at a time).

12. Repeat until vocabulary reaches target size.

5. Tokenization Comparison Table

Property	BPE	WordPiece	Unigram LM
Direction		Grows vocab (expand)	Grows vocab (expand)
Shrinks vocab (prune)			
Merge criterion		Frequency	Likelihood (PMI)
Minimize log-loss			
Test time	Apply merge rules in order	Match longest subword in vocab	Viterbi over vocab

Unknown char <UNK>	Unknown char → <UNK> Multiple fallback segmentations	Unknown subword → WHOLE WORD
-----------------------	---	------------------------------

Used by GPT, GPT-2, RoBERTa BERT, mBERT, DistilBERT T5, XLNet, ALBERT (SentencePiece)

6. Sample Questions & Solutions

[EASY] What is the main advantage of sub-word tokenization over word-level tokenization?

ANS: Sub-word tokenization handles OOV words by splitting them into known sub-units. It achieves a balanced vocabulary size (not too large like word-level, not too small like character-level) and can handle morphological variation (e.g., 'running' → 'run' + '##ning').

[MEDIUM] BPE vs WordPiece: How do their merge criteria differ, and how do they handle unknown tokens at test time?

ANS: BPE: merges the pair with the highest raw frequency. At test time: apply merge rules in sequence; unknown characters → <UNK>. WordPiece: merges the pair with the highest PMI score = $\text{freq}(AB) / (\text{freq}(A) \times \text{freq}(B))$. At test time: no merge rules stored; finds longest vocabulary match left-to-right; if any sub-token is missing, the WHOLE WORD becomes <UNK>.

[HARD] Explain how Unigram LM tokenization uses the EM algorithm and how it decides which tokens to remove.

ANS: E-step: For each word, use Viterbi to find the best segmentation (highest probability under current unigram model). Compute total log-likelihood loss. M-step: Tentatively remove each vocabulary token; re-compute the loss. The token whose removal causes the smallest increase in loss (least important token) is removed. Repeat until vocabulary shrinks to target size. This is EM because E finds best segmentations given current vocab, M updates vocab to maximize likelihood.

7. Quick Revision — Topic I

► **INFO:** BPE=frequency merge, grows vocab, merge rules at test time, unknown char→UNK. WordPiece=PMI merge, grows vocab, longest match at test time, unknown subword→WHOLE WORD UNK. ULM=shrinks vocab, EM algorithm, multiple candidate segmentations. GPT=BPE, BERT=WordPiece, T5=SentencePiece(ULM).

TOPIC J: ADVANCED PROMPTING TECHNIQUES & PROMPT SENSITIVITY (POSIX)

1. Chain-of-Thought (CoT) Prompting

Standard few-shot prompts only show question → answer pairs. For complex reasoning tasks (math, logic), the model doesn't see the reasoning and makes errors. CoT prompting includes intermediate reasoning STEPS in the few-shot examples.

Type	How	When to Use
Few-Shot CoT	Provide 2-8 (question, reasoning chain, answer) examples in the prompt	When you have labeled reasoning examples
Zero-Shot CoT	Append 'Let's think step by step.' to the question — no examples needed	When no examples exist; works well on large models

► **EXAM TIP:** 'Let's think step by step' is the canonical zero-shot CoT trigger phrase. CoT emerged as a key technique around 2022 (Wei et al., Google Brain).

2. Self-Consistency

Problem with CoT: stochastic models produce different reasoning chains for the same question. A single chain might be wrong.

- Generate MULTIPLE reasoning chains for the same question (N different chains due to stochasticity)
- Each chain arrives at an answer. Take the MAJORITY VOTE
- The majority answer is more reliable than any single chain

► **EXAM TIP:** Self-consistency exploits the stochastic nature of LLMs — outputs are sampled from a probability distribution, not deterministic.

3. Tree of Thoughts (ToT)

Extends CoT to a branching search structure. Instead of one linear chain, ToT explores multiple possible reasoning paths simultaneously — like branch-and-bound algorithms.

Component	Function
Analogy	
Branching Module	Decides when to split into multiple sub-steps (explore alternatives)
Scoring Module	Evaluates quality/promise of each branch (positive/negative scores)
	Tree branching Heuristic function

Backtracking Module When branch is scored poorly, backtracks to previous node Pruning

► **INFO:** ToT uses separate LLM calls for branching, scoring, and backtracking — each is its own module. Classic example: 'Game of 24' (4 numbers → reach 24 using arithmetic).

4. Graph of Thoughts (GoT)

Further extension: thought chains are connected as a directed graph. Thoughts from multiple branches can be AGGREGATED (combined).

Method	Structure	Key Feature	Complexity
Standard Prompt	Q → A		Single forward pass
Low			
Chain-of-Thought	Q → Steps → A		Linear reasoning chain
Low			
Self-Consistency	Multiple CoT + voting		Majority vote reduces variance
Medium			
Tree of Thoughts	Branching tree		Branch + score + backtrack
High			

Graph of Thoughts Connected graph Merge and reuse thoughts Very High

5. Prompt Sensitivity & POSIX Index

Accuracy alone is insufficient to evaluate LLMs. A model that gives correct answers to some phrasings but wrong answers to paraphrases is UNRELIABLE. Prompt sensitivity = how much output changes when the prompt is slightly rephrased while preserving intent.

► **EXAM TIP:** model with HIGH sensitivity (high POSIX score) is BAD. You want LOW sensitivity = HIGH robustness.

5.1 POSIX — Prompt Sensitivity Index

POSIX captures four criteria to quantify prompt sensitivity:

Criterion	What It Measures
High Value = ?	

Response Diversity Number of unique responses across prompt variations More sensitive (bad)

Distribution Entropy Entropy of response distribution (2,2,2,2,3 vs 2,3,4,5,6) More sensitive (bad)

Semantic Coherence	Cosine similarity between response embeddings
HIGH similarity = LESS sensitive (good)	
Confidence Variance	Variance in log-likelihood of generated responses
More variance = more sensitive (bad)	

5.2 Key Research Findings

- Adding few-shot examples GENERALLY reduces sensitivity (improves robustness)
- Instruction-tuning does NOT guarantee lower sensitivity — no clear pattern
- Larger models are not always less sensitive — depends on the task
- Sensitivity must be reported ALONGSIDE accuracy for comprehensive evaluation

► **EXAM TIP:** *refer Model B (75% accuracy, 20% sensitivity) over Model A (85% accuracy, 60% sensitivity) — sensitivity is as important as accuracy for production reliability.*

6. Sample Questions & Solutions

[EASY] What is the difference between zero-shot CoT and few-shot CoT?

ANS: Few-shot CoT: provides 2-8 complete examples of (question + reasoning steps + answer). The model learns the reasoning format from examples. Zero-shot CoT: no examples provided. Simply append 'Let's think step by step.' — the model generates its own reasoning chain. Zero-shot CoT works well on large models even without examples.

[MEDIUM] Explain the three modules in Tree of Thoughts and how they work together.

ANS: Branching module: generates multiple possible next reasoning steps (explores alternatives). Scoring module: evaluates each branch using a scoring function or LLM judge — assigns positive/negative scores. Backtracking module: when a branch receives a low score (dead end), backtracks to the previous node and explores other branches. These work together in a branch-and-bound search for the best reasoning path.

[HARD] What is POSIX and what are its four evaluation criteria? Why is accuracy alone insufficient for evaluating LLMs?

ANS: POSIX measures how sensitive a model is to intent-preserving prompt variations. Four criteria: (1) Response Diversity: number of unique responses across variations — more unique = more sensitive. (2) Distribution Entropy: entropy of response distribution — higher = more sensitive. (3) Semantic Coherence: cosine similarity of response embeddings — lower similarity = more sensitive. (4) Confidence Variance: variance in log-likelihood — higher = more sensitive. Accuracy is insufficient because a model can be accurate on one phrasing but wrong on a paraphrase — brittle and unreliable in production.

7. Quick Revision — Topic J

► **INFO:** *CoT: few-shot with reasoning steps OR zero-shot 'Let's think step by step'. Self-Consistency: multiple chains + majority vote. ToT: branch+score+backtrack (branch-and-bound). GoT: connected graph, aggregate thoughts. POSIX: lower=better; few-shot reduces sensitivity; bigger model not always less sensitive.*

TOPIC K: CODE WALKTHROUGHS — 4 LAB NOTEBOOKS

This section extracts key implementation insights from the 4 course Jupyter notebooks. Understanding the code logic helps answer architecture, hyperparameter, and training strategy questions.

K.1 Attention Mechanism — NumPy Implementation (attention.ipynb)

This notebook implements scaled dot-product attention, self-attention, and positional encoding from scratch using NumPy — making every step transparent.

K.1.1 Numerically Stable Softmax

Softmax Implementation

```
def softmax(x, axis=-1):
    x_shifted = x - np.max(x, axis=axis, keepdims=True) # subtract max to
    prevent overflow
    exp_x = np.exp(x_shifted)
    return exp_x / np.sum(exp_x, axis=axis, keepdims=True)
```

Result sums to 1.0 along specified axis (the attention weight distribution)

K.1.2 Scaled Dot-Product Self-Attention

Self-Attention: $Q=K=V=X$ (same sequence)

```
tokens = ["the", "cat", "sat", "on", "the", "mat"]
X = np.array([...]) # token embeddings, shape [6, d]
Q, K, V = X, X, X # In real Transformers: Q=X@Wq, K=X@Wk, V=X@Wv
d_k = Q.shape[1] # embedding dimension
scores = Q @ K.T # raw dot products, shape [seq_len, seq_len]
scaled = scores / np.sqrt(d_k) # scale to prevent vanishing gradients
weights = softmax(scaled, axis=-1) # attention weights, rows sum to 1
context = weights @ V # weighted sum of VALUES, shape [seq_len, d]
```

► **EXAM TIP:** *n* self-attention: Q, K, V all come from the SAME sequence. Output = weighted sum of VALUES. Scale by $\sqrt{d_k}$ before softmax. Self-attention is PERMUTATION INVARIANT.

K.1.3 Sinusoidal Positional Encoding

Positional Encoding Implementation

```
def positional_encoding(seq_len, d_model):
    pe = np.zeros((seq_len, d_model))
    positions = np.arange(seq_len)[:, np.newaxis] # shape [seq_len, 1]
    div_terms = np.exp(np.arange(0, d_model, 2) * (-np.log(10000.0) / d_model))
    pe[:, 0::2] = np.sin(positions * div_terms) # even dimensions → sin
    pe[:, 1::2] = np.cos(positions * div_terms) # odd dimensions → cos
    return pe # shape [seq_len, d_model]
# Final input = word_embedding + positional_encoding (element-wise addition)
```

► **INFO:** $pe[:, 0::2]$ = every 2nd column from 0 (even: 0,2,4...) → sin. $pe[:, 1::2]$ = every 2nd column from 1 (odd: 1,3,5...) → cos.

K.2 LSTM for Next-Word Prediction (lstm_project.ipynb)

Task: Train a stacked LSTM model to predict the next word in a sequence. Applied to a FAQ corpus.

K.2.1 Data Preparation Pipeline

Text Preprocessing with n-gram sequences

Step 1: Tokenize

```
tokenizer = Tokenizer()
```

tokenizer.fit_on_texts([faqs]) # builds word_index dict

Step 2: Create sliding-window n-gram sequences

```
for sentence in faqs.split('\n'):
    tokens = tokenizer.texts_to_sequences([sentence])[0]
    for i in range(1, len(tokens)):
        input_sequences.append(tokens[:i+1]) # all n-grams
```

Step 3: Pad sequences (PRE-padding — zeros at START)

```
padded = pad_sequences(input_sequences, maxlen=max_len, padding='pre')
# Step 4: X=all but last token, y=last token (one-hot)
X, y = padded[:, :-1], padded[:, -1]
y = to_categorical(y, num_classes=vocab_size)
```

► **EXAM TIP:** *-padding (zeros at start): ensures the LAST token is the most recent word, which the LSTM processes last — its hidden state is most meaningful for prediction.*

K.2.2 Stacked LSTM Architecture

Stacked LSTM Model

```
model = Sequential([
    Embedding(vocab_size, 100, input_length=max_len-1),
    LSTM(150, return_sequences=True), # return_sequences=True MANDATORY for
    stacking!
    LSTM(150), # last LSTM: return_sequences=False (default)
    Dense(vocab_size, activation='softmax')
])
```

```
model.compile(loss='categorical_crossentropy', optimizer='adam')
```

► **EXAM TIP:** *return_sequences=True is MANDATORY for all LSTMs except the last when stacking. Without it, next LSTM gets 2D input (batch, features) instead of 3D (batch, timesteps, features).*

K.2.3 Text Generation (Inference)

Greedy Inference Loop

```
text = "what is the fee"
for i in range(10): # generate 10 more words
    tokens = tokenizer.texts_to_sequences([text])[0]
    padded = pad_sequences([tokens], maxlen=56, padding='pre')
    pos = np.argmax(model.predict(padded)) # greedy: pick highest probability
    for word, index in tokenizer.word_index.items():
        if index == pos: text += " " + word
```

► **EXAM TIP:** *Greedy decoding: always picks highest probability next word. Alternative: Beam Search (keep top-k hypotheses for better sequences).*

K.3 Simple RNN for Sentiment Analysis (sentiment_analysis_simplernn.ipynb)

Task: Sentiment classification (positive/negative) on IMDB dataset using SimpleRNN.

SimpleRNN Model for Binary Sentiment

```
(X_train, y_train), (X_test, y_test) = imdb.load_data()
X_train = pad_sequences(X_train, padding='post', maxlen=50)
model = Sequential([
    Embedding(input_dim=10000, output_dim=2, input_length=50),
```

```
# Embedding: maps token indices → dense 2D vectors
SimpleRNN(32, return_sequences=False), # 32 hidden units, only final state
Dense(1, activation='sigmoid') # Binary output: 0=neg, 1=pos
```

))

```
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['acc'])
history = model.fit(X_train, y_train, epochs=5, validation_data=(X_test,
y_test))
```

NOTE: SimpleRNN suffers from vanishing gradients for long sequences. For short sequences (50 words) it can work. Embedding(10000, 2, 50): vocab=10k; each word→2D vector; input length=50.

K.4 Image Captioning — CNN + LSTM (epgd_Image_captioning.ipynb)

Task: Generate natural language captions for images (Flickr8k: 8,000 images × 5 captions each = 40,000 pairs).

Architecture: CNN encoder + LSTM decoder.

K.4.1 Architecture Overview

Component	Architecture	Key Detail
Encoder (CNN)	ResNet-50 (pretrained ImageNet)	Final FC layer removed; ALL params frozen; new linear projection added

Decoder (RNN)	LSTM (hidden_size=512)	Takes image feature + previous word → generates next word
---------------	------------------------	---

Training	Teacher Forcing	Feed ground-truth previous word, not model's own prediction
----------	-----------------	---

Special tokens	<start>, <end>, <unk>, <pad>	<start>/<end> mark caption boundaries; <unk> for rare words
----------------	------------------------------	---

K.4.2 Frozen CNN Encoder

EncoderCNN — Transfer Learning

class EncoderCNN(nn.Module):

```
def __init__(self, embed_size):
    resnet = models.resnet50(weights=ResNet50_Weights.IMAGENET1K_V2)
```

Freeze ALL ResNet parameters — no fine-tuning

```
for param in resnet.parameters():
    param.requires_grad = False # <-- KEY: transfer learning
```

Remove last classification layer; add projection to embed_size

```
modules = list(resnet.children())[:-1] # remove final FC
self.resnet = nn.Sequential(*modules)
self.linear = nn.Linear(resnet.fc.in_features, embed_size)
self.bn = nn.BatchNorm1d(embed_size) # stabilize features
```

► **EXAM TIP:** Freezing ResNet (requires_grad=False) = Transfer Learning. Only the linear projection and LSTM decoder are trained. This is efficient and prevents catastrophic forgetting.

K.4.3 Teacher Forcing vs Autoregressive Inference

Training (Teacher Forcing)	Inference (Autoregressive)
Input at step t	Ground truth word t-1
	Model's own prediction at t-1
Advantage	Stable gradients, fast convergence
	Realistic generation

Problem	Exposure bias (train/test mismatch)	Errors compound over long sequences
---------	-------------------------------------	-------------------------------------

K.4.4 Key Hyperparameters

Hyperparameter Value Controls

embed_size 256 Dimension of image features and word embeddings

hidden_size 512 LSTM hidden state dimension

num_epochs 5 Training passes through dataset

learning_rate 0.0001 Adam optimizer step size (small for stability)

batch_size 64 Pairs per gradient update

train/val/test split 70/10/20 Based on IMAGE IDs (not individual captions – prevents leakage!)

► **EXAM TIP:** Split is on IMAGE IDs: same image cannot appear in both train and test even with 5 captions. This prevents data leakage.

5. Sample Questions — Code Walkthroughs

[EASY] Why is return_sequences=True required for the first LSTM in a stacked LSTM model?

ANS: When stacking two LSTM layers, the first LSTM must output a sequence (hidden state at EVERY timestep) for the second LSTM to process. With return_sequences=False (default), only the FINAL hidden state is returned – a single vector, not a sequence. The second LSTM expects 3D input (batch, timesteps, features), so return_sequences=True is mandatory for all LSTMs except the last one.

[MEDIUM] In the image captioning model, why is the CNN (ResNet-50) frozen during training?

ANS: The CNN is frozen (requires_grad=False) because: (1) ResNet-50 is already pretrained on ImageNet and extracts high-quality visual features. (2) Fine-tuning alongside the LSTM would require significantly more compute and data. (3) Freezing prevents catastrophic forgetting of ImageNet features. Only the linear projection and LSTM decoder are trained – efficient Transfer Learning.

[HARD] Explain teacher forcing in image captioning: what problem it solves, and what new problem it creates.

ANS: Teacher forcing: during training, the LSTM decoder receives GROUND TRUTH previous word at each step, regardless of model prediction. Problem solved: without teacher forcing, early prediction errors cascade – wrong word at step t causes wrong input at step t+1, making gradients unstable and training slow. Problem created (exposure bias): during inference, the model uses its OWN predictions as the next input (no ground truth). The model was never trained on this scenario, so small errors compound over long sequences. Mitigation: scheduled sampling (gradually replace ground truth with model predictions during late training).

6. Quick Revision — Topic K

► **INFO:** Attention notebook: $\text{scores} = Q @ K.T / \text{vd_k} \rightarrow \text{softmax} \rightarrow \text{output} = \text{weights} @ V$. Positional encoding: $\sin/\cos$, add to embeddings. LSTM notebook: pre-padding, return_sequences=True for stacking, greedy inference. SimpleRNN: Embedding $\rightarrow$ RNN(32) $\rightarrow$ Dense(sigmoid), binary_crossentropy. Image captioning: ResNet-50 frozen (transfer learning), LSTM decoder, teacher forcing, split on image IDs.

MASTER FORMULA SHEET — ALL TOPICS (A through K)

A: N-grams & Smoothing

Formula

When to Use

$P(w_{\square} | w_{\square-1}) = \text{count}(w_{\square-1}, w_{\square}) / \text{count}(w_{\square-1})$
probability (MLE)

Bigram

$P_add1(w v) = (c(v,w)+1) / (c(v)+ V)$ smoothing	Laplace/Add-1
$c^* = (c+1) \cdot N_{\{c+1\}} / N_c$ modified count	Good-Turing
$PP(W) = P(w_1 \dots w_n)^{-1/N}$ lower = better	Perplexity -
$PP = 2^H$ where $H = -1/N \cdot \sum \log_2 P(w_i ctx)$ cross-entropy	Perplexity from
$P_KN(w_i w_{i-1}) = \max(c(w_{i-1}, w_i) - D, 0) / c(w_{i-1}) + \lambda \cdot P_cont(w_i)$ smoothing	Kneser-Ney

B: RNN / LSTM / GRU

Formula What it is

$h_t = \tanh(W_h h_{t-1} + W_e x_t + b)$	Vanilla RNN hidden state
$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$	LSTM forget gate
$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$	LSTM input gate
$C_t = f_t \odot C_{t-1} + i_t \odot \tanh(W_c[h_{t-1}, x_t])$ FORMULA)	LSTM cell state update (KEY FORMULA)
$h_t = o_t \odot \tanh(C_t)$	LSTM hidden state output
$z_t = \sigma(W_z[h_{t-1}, x_t])$ $r_t = \sigma(W_r[h_{t-1}, x_t])$	GRU update and reset gates
$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tanh(W[h_{t-1}, x_t])$	GRU hidden state

C: Attention & Seq2Seq

Formula What it is

$e_{i,j} = s_i \cdot h_j$	Attention score (query-key dot product)
$\alpha_{i,j} = \text{softmax}(e_{i,j})$ distribution)	Attention weights (probability distribution)
$a_i = \sum_j \alpha_{i,j} \cdot v_j$ (critical!)	Attention output = weighted sum of VALUES
$\text{Score}(BM, y_1 \dots y_n) = \sum \log P(y_i) / t$	Length-normalized beam search score

D: Transformer & Self-Attention

Formula What it is

$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$ (Transformer)	Scaled dot-product attention
$\text{MultiHead} = \text{Concat}(\text{head}_i) W_o$	Multi-head attention
$\text{PE}(p, 2i) = \sin(p / 10000^{2i/d})$ dimensions)	Positional encoding (even dimensions)
$\text{PE}(p, 2i+1) = \cos(p / 10000^{2i/d})$	Positional encoding (odd dimensions)
$\text{FFN}(x) = \max(0, xW_1 + b_1) W_2 + b_2$	Feed-forward network (ReLU)
$\text{LayerNorm}(x) = \gamma(x - \mu) / \sigma + \beta$	Layer normalization
$\text{Out} = \text{LayerNorm}(x + \text{Sublayer}(x))$	Residual connection + norm

G: RLHF & Alignment

Formula What it is

$P(y_1 \succ y_2 x) = \sigma(r(x, y_1) - r(x, y_2))$ model (simplified)	Bradley-Terry preference
$L(RM) = -\log \sigma(r_\theta(x, y^+) - r_\theta(x, y^-))$	Reward model loss (maximize)
$\max_\theta E[r(x, y) - \beta \cdot \log(\pi_\theta(y x) / \pi_{\text{ref}}(y x))]$	RLHF objective (reward - KL penalty)

$\nabla J \approx E[A(s,a) \cdot \nabla \log \pi_\theta(a s)]$ advantage	Policy gradient with advantage
$A(s,a) = Q(s,a) - V(s)$ quality of action)	Advantage function (relative quality of action)
$r_\square = \pi_\theta(a_\square s_\square) / \pi_{old}(a_\square s_\square)$	PPO importance ratio
$L_{PPO} = E[\min(r_\square A_\square, \text{clip}(r_\square, 1-\epsilon, 1+\epsilon) A_\square)]$	PPO-clip objective

I: Tokenization

Formula What it is

BPE: $\text{argmax}_{\{A,B\}} \text{freq}(A,B)$ BPE merge criterion: highest raw frequency

WordPiece: $\text{score}(A,B) = \text{freq}(A,B) / (\text{freq}(A) \times \text{freq}(B))$ criterion: PMI-like score	WordPiece merge
ULM: $P(t_1, t_2, \dots, t_\square) = \prod_i P(t_i)$ segmentation probability	Unigram LM
ULM Loss = $\sum_w \text{freq}(w) \times -\log P(\text{best_seg}(w))$ for ULM pruning	Log-likelihood loss

J: POSIX Sensitivity

Formula What it is

POSIX = measure of variance in $P_M(y x_i)$ across prompt variants x_i Prompt sensitivity index	
Higher POSIX $\rightarrow$ more sensitive $\rightarrow$ less robust (bad) Interpretation rule	
Semantic coherence = $\text{cosine_similarity}(\text{embed}(\text{response}_i), \text{embed}(\text{response}_j))$ Response consistency measure	
Entropy = $-\sum P(\text{response}) \log P(\text{response})$ Response distribution entropy	

COMMON EXAM MISTAKES & HOW TO AVOID THEM

Mistake	Correct Understanding	Exam Ref
GloVe uses negative sampling like Word2Vec sampling Q1.1 FALSE	GloVe uses global co-occurrence matrix — NO negative sampling	

Larger beam size $\rightarrow$ smaller probability $\rightarrow$ HIGHER probability (more search = better solution)	Larger beam size Q1.5 FALSE
--	--------------------------------

RAG increases hallucination RAG REDUCES hallucination (grounded in retrieved passages)
Q1.6 FALSE

Attention output = weighted sum of queries weighted sum of VALUES: $a_\square = \sum \alpha_\square, \square \cdot v_\square$	Output = Q1.12 $\rightarrow$ C
Encoder-only model works without positional encoding (GPT) can work without PE — causal masking implies order	DECODER-only Q1.2 $\rightarrow$ B

LSTM eliminates vanishing gradient LSTM REDUCES (not eliminates) vanishing gradient

Concept

Reflexion updates model weights verbal self-reflection only — NO weight updates	Reflexion uses Q1.19 $\rightarrow$ B
ReAct decouples planning from execution planning. ReAct interleaves reasoning with actions	ReWOO decouples Q1.17 $\rightarrow$ C
Self-Instruct uses Instruction Evolver uses Instruction Evolver (constraints/deepening)	EVOL-INSTRUCT Q1.14 $\rightarrow$ C

BPE merges based on likelihood like WordPiece BPE merges based on RAW FREQUENCY. WordPiece uses
PMI-like score Topic I

WordPiece stores merge rules like BPE
longest match Topic I

WordPiece does NOT store merge rules — only vocab, uses

Unknown subword in WordPiece → just that subword is <UNK>	Unknown subword
in WordPiece → WHOLE WORD becomes <UNK>	Topic I CRITICAL
Unigram LM expands vocabulary like BPE	Unigram LM
SHRINKS vocabulary — starts large and prunes	Topic I
Higher POSIX = more robust model	Higher POSIX =
MORE SENSITIVE = LESS robust. Lower POSIX is better	Topic J
return_sequences doesn't matter for stacked LSTM	
return_sequences=True MANDATORY for all but last LSTM in stack	
Topic K	

PREDICTED EXAM QUESTIONS WITH COMPLETE SOLUTIONS

PART 2 — DEEP DIVE: TRANSFORMER & SELF-ATTENTION

Source: DL_Part2_SecondHalf_NLP_LLM_RLHF.docx — This has 3× more detail on the Transformer than the study notes. Read after Part 1 Topic D.

Lectures: Lec-5, Lec-6 | Weight: ~15% — HIGHEST PRIORITY | Key: Q/K/V math, masking, positional encoding, architecture

D.1 Motivation: Why Self-Attention?

- INTUITION: The Key Insight
- If attention lets decoder attend to all encoder states, why not let EACH TOKEN attend to ALL OTHER TOKENS in the same sequence? This is self-attention — remove the recurrent connections entirely and process all tokens IN PARALLEL.

- Problems with RNN-based attention:
 1. Sequential: must compute h_1 before h_2 before $h_3 \rightarrow$ cannot parallelize
 2. Even with attention, gradient still flows through recurrent links
 3. Long-range dependencies: h^T depends on h_1 through many multiplicative steps
- Self-attention solution:
 - Remove recurrent links entirely
 - Each token directly attends to ALL other tokens in $O(1)$ steps
 - Complexity: $O(n^2 \cdot d)$ — quadratic in sequence length, but PARALLELIZABLE
 - Training: can use modern GPUs/TPUs at full capacity

D.2 Scaled Dot-Product Self-Attention — COMPLETE DERIVATION

- SETUP: Given input sequence $X = [x_1, \dots, x_n]$ where $x_i \in \mathbb{R}^d$
- STEP 1 — Generate Q, K, V from input:
 - $Q = X \cdot W_Q$ $Q \in \mathbb{R}^{n \times d_k}$ [queries — 'what am I looking for?']
 - $K = X \cdot W_K$ $K \in \mathbb{R}^{n \times d_k}$ [keys — 'what do I advertise?']
 - $V = X \cdot W_V$ $V \in \mathbb{R}^{n \times d_v}$ [values — 'what do I actually contain?']
 - $W_Q \in \mathbb{R}^{d \times d_k}$, $W_K \in \mathbb{R}^{d \times d_k}$, $W_V \in \mathbb{R}^{d \times d_v}$ (learnable weight matrices)
- STEP 2 — Compute raw attention scores ($Q \cdot K^T$):
 - $E = Q \cdot K^T \in \mathbb{R}^{n \times n}$ where $E_{ij} = q_i \cdot k_j$ (similarity of token i to token j)
- STEP 3 — Scale by $\sqrt{d_k}$ (prevent vanishing gradients in softmax):
 - $E_{\text{scaled}} = E / \sqrt{d_k}$
 - WHY $\sqrt{d_k}$ scaling? If d_k is large, dot products get large $\rightarrow$ softmax saturates
 - $\rightarrow$ gradients become tiny. Dividing by $\sqrt{d_k}$ keeps variance ≈ 1 .
- STEP 4 — Optional: Apply MASK (set future positions to $-\infty$ for decoder)
 - $E_{\text{masked}} = E_{\text{scaled}} + M$ where $M_{ij} = -\infty$ if $j > i$, else 0
- STEP 5 — Softmax to get attention weights:
 - $A = \text{softmax}(E_{\text{masked}}) \in \mathbb{R}^{n \times n}$ where $A_{ij} = \exp(E_{ij}) / \sum_k \exp(E_{ik})$

- STEP 6 — Compute weighted sum of values:
- Output = $A \cdot V \in \mathbb{R}^{n \times d_v}$
- COMPLETE FORMULA (single head):
- $\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$

► **EXAM TIP:** Know this formula cold. Exam asks for Q, K, V formulation repeatedly. Also: $e^{(-\infty)} = 0$ after softmax
 → masked positions get weight=0 → model cannot attend to future positions.

D.3 Multi-Head Attention

- INTUITION: Why Multiple Heads?
- Like CNN filters detecting different visual features, each attention head learns to attend to different types of relationships: syntactic dependencies, semantic similarity, coreference, etc. Single head softmax often focuses on just one thing.

- Multi-Head Attention with h heads:
- For each head $i = 1, \dots, h$:
- $\text{head}_i = \text{Attention}(X \cdot WQ_i, X \cdot WK_i, X \cdot WV_i)$
- where $WQ_i \in \mathbb{R}^{d \times (d/h)}$, $WK_i \in \mathbb{R}^{d \times (d/h)}$, $WV_i \in \mathbb{R}^{d \times (d/h)}$
- Concatenate all heads:
- $\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W^O$
- where $W^O \in \mathbb{R}^{(h \cdot d/h) \times d} = \mathbb{R}^{d \times d}$ (output projection back to d dims)
- Tensor shapes (n = sequence length, d = model dim, h = #heads):
- Each head operates in subspace of dim d/h
- Concat of h heads → $h \times (d/h) = d$ → W^O projects back to d
- Standard values (original Transformer): d=512, h=8, $d_k=d_v=512/8=64$
- (BERT-Base): d=768, h=12, dk=64
- (GPT-2 small): d=768, h=12, dk=64

D.4 Positional Encoding

- INTUITION: The Problem
- Self-attention is PERMUTATION INVARIANT — 'The cat sat' and 'sat cat The' give same attention (if we ignore word identity). We need to inject position information. But unlike RNN, there are no recurrent connections to carry position info automatically.

- Sinusoidal Positional Encoding (original Transformer paper):
- $PE(pos, 2i) = \sin(pos / 10000^{(2i/d)})$
- $PE(pos, 2i+1) = \cos(pos / 10000^{(2i/d)})$
- $pos = \text{position } (0, 1, 2, \dots)$
- $i = \text{dimension index } (0 \text{ to } d/2-1)$
- $d = \text{model dimension (e.g., 512)}$
- Input to Transformer: $X_i = E_i + PE(i)$ [embedding + positional encoding]
- Properties:
 - Unique for each position
 - $|PE(pos)| \leq 1$ (bounded, won't dominate embeddings)
 - Relative position $PE(pos+k)$ is a linear function of $PE(pos)$ → model can learn
 - Works for sequences longer than seen during training
- Alternative: Learned positional embeddings (GPT-2, BERT)
- → simply learns a matrix $PE \in \mathbb{R}^{(\text{max_len} \times d)}$ during training

D.5 Complete Transformer Architecture

- ENCODER BLOCK ($\times N$, e.g., $N=6$ in original paper):
 1. Multi-Head Self-Attention (all tokens attend to all tokens — UNMASKED)
 2. Add & Norm: $x \leftarrow \text{LayerNorm}(x + \text{Attention}(x))$ [residual connection]
 3. Feed-Forward Network: $\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$ [position-wise]
 4. Add & Norm: $x \leftarrow \text{LayerNorm}(x + \text{FFN}(x))$
- FFN dimensions: $d_{\text{model}} \rightarrow d_{\text{ff}} \rightarrow d_{\text{model}}$ where $d_{\text{ff}} = 4 \times d_{\text{model}}$ typically
(project UP then project DOWN: $512 \rightarrow 2048 \rightarrow 512$ in original Transformer)
- DECODER BLOCK ($\times N$):
 1. MASKED Multi-Head Self-Attention (token i only sees positions $\leq i$)
 2. Add & Norm
 3. CROSS-ATTENTION: Q from decoder, K, V from ENCODER output
→ decoder conditions on full source sequence
 4. Add & Norm
 5. Feed-Forward Network
 6. Add & Norm
- Final: Linear projection → softmax → probability over vocabulary

► **EXAM TIP:** 3 types of attention in Transformer: (1) Encoder self-attention [unmasked] (2) Decoder masked self-attention [causal mask] (3) Decoder cross-attention [decoder queries encoder K, V]. Know all 3.

D.5.1 Why Residual Connections + Layer Norm?

- Residual (Skip) connection: $x_{\text{out}} = \text{LayerNorm}(x + \text{Sublayer}(x))$
- Benefits of residual:
 - Gradient highway: $\partial L / \partial x = \partial L / \partial x_{\text{out}} \times (1 + \partial \text{Sublayer} / \partial x)$

- - The '1' ensures gradient always flows → no vanishing
- - Allows stacking 6, 12, 24+ layers without degradation
- Layer Normalization (applied AFTER residual in original; BEFORE in modern):
- $\text{LN}(x) = \gamma \cdot (x - \mu) / \sqrt{\sigma^2 + \epsilon} + \beta$ [per-sample, across feature dimension]
- μ = mean over d features, σ^2 = variance over d features
- γ, β = learned per-feature scale/shift parameters
- Why LayerNorm (not BatchNorm) in Transformers?
- - Sequences have variable length → batch statistics are unstable
- - LayerNorm works with batch size 1 and variable lengths

+-----+

D.6 Masked Self-Attention — Causal Masking in Decoder

+-----+

- During TRAINING on 'I love cats':
- Decoder input: [`<s>`, I, love, cats]
- Decoder output: [I, love, cats, `</s>`]
- To prevent 'cheating' (using future tokens), apply causal mask:
- Attention score matrix E (4×4 for 4 tokens):
- $E_{\text{masked}}[i,j] = E[i,j]$ if $j \leq i$
- $= -\infty$ if $j > i$
- After softmax: $\exp(-\infty) = 0 \rightarrow$ future positions get 0 attention weight
- In practice: add lower-triangular mask matrix M before softmax
- $M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}$, [position 1 can only see itself]
- [0, 0, -∞, -∞], [position 2 sees positions 1,2]
- [0, 0, 0, -∞], [position 3 sees 1,2,3]
- [0, 0, 0, 0]] [position 4 sees all]
- During INFERENCE: generate one token at a time; no mask needed
- (future tokens don't exist yet → naturally causal)

+-----+

D.7 Transformer Parameter Count

+-----+

- Per encoder/decoder layer:
- Multi-Head Attention: 4 matrices $\times d^2 = 4d^2$ [WQ, WK, WV, WO]
- FFN: $2d \times d_{\text{ff}} + d_{\text{ff}} \times d = 2d \times d_{\text{ff}}$ (typically $d_{\text{ff}}=4d$) = $8d^2$
- LayerNorms: $2 \times (2d) = 4d$ (2 LNs per block, each has γ and β of dim d)
- BERT-Base: $d=768, h=12, L=12$ layers
- Attention per layer: $4 \times 768^2 = 2,359,296$ params
- FFN per layer: $2 \times 768 \times 3072 = 4,718,592$ params
- Per layer total: $\sim 7.08\text{M} \times 12 \text{ layers} = \sim 85\text{M}$
- Embedding: $30522 \times 768 = \sim 23\text{M}$
- Total: $\sim 110\text{M}$ parameters
- GPT-2 small: $d=768, h=12, L=12$, same architecture → $\sim 117\text{M}$ params

+-----+

PART 3 — DEEP DIVE: RLHF & ALIGNMENT

Source: *DL_Part2_SecondHalf_NLP_LLM_RLHF.docx* — 5× more detail than study notes. Covers complete RL formulation, reward model, PPO objective, KL regularisation, DPO, and alternatives.

Lectures: Lec-9, Transcripts Lec-24,25 | Weight: ~15% — HIGH PRIORITY | Key: RLHF pipeline, Bradley-Terry, PPO objective, KL reg

G.1 Why IFT is Not Enough — Alignment Problem

- IFT makes model follow instructions, but cannot prevent harmful outputs.
- Example failure case:
 - User: 'What is the best way to lose weight quickly?'
 - IFT model: 'Stop eating entirely for a few days.' [dangerous but instruction-following]
 - Aligned model: 'Reduce calorie intake, increase exercise...' [safe and helpful]
- Instruction tuning: CAN make model follow the instruction format
- Alignment (RLHF): PREVENTS model from generating unsafe/unethical outputs even when the model 'thinks' it's a correct answer
- Goal of alignment: model outputs should be helpful, harmless, and honest (3H)

G.2 LLM as a Reinforcement Learning Agent

- RL Formulation for LLMs:
- Policy $\pi_{\theta}(a|s)$ = the LLM, parameterized by θ
- Action a_t = a single token generated at step t
- State s_t = [prompt tokens, all previously generated tokens up to t]
- $s_t = [x_1, \dots, x_t, y_1, \dots, y_{t-1}]$ (prompt x + generated y)
- State transition: $s_{t+1} = \text{append}(s_t, a_t)$ [deterministic]
- Reward: $r_t = 0$ for intermediate tokens
- $r_T = r(x, y)$ from reward model [only at end of generation]
- Environment: abstract (text input + generated output + feedback)
- NOT like explicit game simulators (Atari) — the 'world' is language itself
- Episode: one prompt $\rightarrow$ generate complete response $\rightarrow$ get reward

► **EXAM TIP:** xam Q6: 'Define Action and State in LLM-RL framework.' Action = single output token. State = prompt + all previously generated tokens. This appeared in the model paper!

G.3 Reward Model

- Goal: Learn a model $r(x, y)$ that predicts human preference score
- x = prompt, y = response

- $r(x, y_1) > r(x, y_2)$ if y_1 is preferred over y_2
- Architecture:
- Base: Transformer Encoder (or same architecture as policy LLM)
- Add: Linear layer on top of average pooling of token embeddings
- Output: scalar reward score $r \in \mathbb{R}$
- Training data: pairwise comparisons (x, y_+, y_-) where y_+ preferred
- Collected from: human annotators comparing model outputs
- y_+ = preferred output, y_- = rejected output
- TRAINING OBJECTIVE — Bradley-Terry Model:
- Maximize: $\sum_{\{x, y_+, y_-\}} \log \sigma(r_\theta(x, y_+) - r_\theta(x, y_-))$
- Equivalently: Minimize: $L = -\log \sigma(r_\theta(x, y_+) - r_\theta(x, y_-))$
- Intuition: push reward gap between preferred and rejected outputs
- $\sigma(\Delta r) \rightarrow$ probability that y_+ is better than y_-

+-----+

► **EXAM TIP:** xam Q1.16 and Q7 (model paper): Reward model objective = Maximize $\log \sigma(r(x, y_1) - r(x, y_2))$. NOT MSE. NOT KL. NOT negative log-likelihood of y . Know this EXACTLY.

G.4 RLHF Objective — The Full Derivation

+-----+

- GOAL: Find policy π_θ that:
- (1) Maximizes expected reward: $E_{y \sim \pi_\theta}[r(x, y)]$ should be HIGH
- (2) Stays close to reference policy π_{ref} : $KL(\pi_\theta || \pi_{\text{ref}})$ should be LOW
- Reference policy π_{ref} : instruction-tuned model (SFT model) — starting point
- Why stay close to π_{ref} ?
- (generate gibberish that tricks reward model into high scores)
- KL Divergence (how different is π_θ from π_{ref}):
- $KL(\pi_\theta || \pi_{\text{ref}}) = E_{y \sim \pi_\theta}[\log(\pi_\theta(y|x)/\pi_{\text{ref}}(y|x))]$
- COMPLETE RLHF OBJECTIVE (Regularized Reward Maximization):
- $\max_{\theta} E_{y \sim \pi_\theta(y|x)} [r(x, y) - \lambda \cdot \log(\pi_\theta(y|x)/\pi_{\text{ref}}(y|x))]$
- θ
- $= \max_{\theta} E_{y \sim \pi_\theta} [r(x, y)] - \lambda \cdot KL(\pi_\theta(y|x) || \pi_{\text{ref}}(y|x))$
- θ
- λ (or β in some papers): KL regularization coefficient
- Large λ : stays close to π_{ref} (safe but limited improvement)
- Small λ : more reward optimization (potentially reward hacking)

+-----+

► **EXAM TIP:** xam Q7 (model paper): Write the RLHF objective. MEMORIZE: $E[r(x, y) - \lambda \cdot \log(\pi_\theta(y|x)/\pi_{\text{ref}}(y|x))]$. The log ratio IS the KL term per sample.

G.5 RL Concepts for LLMs

G.5.1 Value Functions

-
- Return $G_t = \sum_{k=0}^{\infty} \gamma^k \cdot r_{t+k}$ [discounted sum of future rewards]
 - γ = discount factor ($0 < \gamma \leq 1$), usually $\gamma=1$ for LLM episodes (finite)
 - Q-function: $Q^\pi(s,a) = E_\pi[G_t \mid s_t=s, a_t=a]$
 - = Expected total reward from state s taking action a , then following π
 - Value function: $V^\pi(s) = E_\pi[G_t \mid s_t=s] = \sum_a \pi(a \mid s) \cdot Q^\pi(s,a)$
 - = Expected total reward from state s , following policy π
 - Advantage function: $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$
 - = How much BETTER is action a compared to average action in state s
 - Positive $A \rightarrow$ action is better than average $\rightarrow$ increase probability
 - Negative $A \rightarrow$ action is worse than average $\rightarrow$ decrease probability
-

G.5.2 REINFORCE Algorithm

-
- REINFORCE (Policy Gradient):
 - $\nabla_\theta J(\theta) = E_{\tau \sim \pi_\theta} [\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot G_t]$
 - G_t = reward-to-go from step t
 - Intuition: increase log-prob of actions that led to high returns
 - decrease log-prob of actions that led to low returns
 - Problems with REINFORCE:
-

G.5.3 PPO — Proximal Policy Optimization

-
- INTUITION: PPO Key Idea
 - Don't let one bad batch destroy your policy. Clip the gradient update so the new policy doesn't deviate too much from old policy. This provides stable, reliable training.
-

-
- Importance Sampling for off-policy gradient:
 - $J(\theta) \approx E_{(s,a) \sim \pi_{\text{old}}} [\pi_\theta(a \mid s) / \pi_{\text{old}}(a \mid s) \cdot A^{\pi_{\text{old}}}(s,a)]$
 - Ratio: $\rho(\theta) = \pi_\theta(a \mid s) / \pi_{\text{old}}(a \mid s)$ [how different is new policy from old]
 - PPO-CLIP objective (what you need for the exam):
 - $L^{\text{CLIP}}(\theta) = E_{(s,a) \sim \pi_{\text{old}}} [\min(\rho(\theta) \cdot A^{\pi_{\text{old}}}(s,a), \text{clip}(\rho(\theta), 1-\epsilon, 1+\epsilon) \cdot A^{\pi_{\text{old}}}(s,a))]$
 - ϵ = clipping parameter (e.g., 0.2)
 - $\text{clip}(\rho, 1-\epsilon, 1+\epsilon)$:
 - if $\rho > 1+\epsilon$: clip to $1+\epsilon$ (don't increase too much even if $A > 0$)
 - if $\rho < 1-\epsilon$: clip to $1-\epsilon$ (don't decrease too much even if $A < 0$)

- The min ensures we take the CONSERVATIVE update
- Why PPO > REINFORCE?

G.6 Complete RLHF Pipeline — Step by Step

- STEP 1: Supervised Fine-Tuning (SFT)
- STEP 2: Reward Model Training
- STEP 3: RL Fine-Tuning with PPO
 - a) Sample prompts from dataset
 - b) Generate responses $y \sim \pi_{\theta}(\cdot|x)$
 - c) Score with reward model: $r(x,y)$
 - d) Add KL penalty: $\text{reward_modified} = r(x,y) - \lambda \cdot \log(\pi_{\theta}(y|x)/\pi_{\text{ref}}(y|x))$
 - e) Estimate advantages using value network $V(s)$
 - f) Update π_{θ} using PPO-CLIP
- STEP 4 (Optional): Iteration

- Q3 [Hard]: Exam Q7: Write the mathematical expression for the Regularized Reward Maximization objective in RLHF. Explain each term and why KL regularization is needed.
- SOLUTION:
 1. OBJECTIVE: $\max_{\pi_{\theta}} \mathbb{E}_{\{x \sim D, y \sim \pi_{\theta}(y|x)\}} [r(x,y) - \lambda \cdot \log(\pi_{\theta}(y|x)/\pi_{\text{ref}}(y|x))]$
 2. Term 1: $r(x,y)$ = reward model score (proxy for human preference) — we want this HIGH
 3. Term 2: $\log(\pi_{\theta}(y|x)/\pi_{\text{ref}}(y|x))$ = per-sample KL divergence term — we want this LOW
 4. λ = regularization coefficient that balances reward vs staying close to reference
 5. Why KL needed: Reward models are imperfect. Policy can 'reward hack' — generate nonsense that tricks the reward model into giving high scores (e.g., repetitive tokens).
 6. KL penalty prevents this by ensuring new policy stays close to sensible reference.
 7. KL also preserves output diversity and prevents mode collapse (only generating one type of response).
 8. Ideal answer format: State formula → define each symbol → explain intuition → explain why KL needed.
- ANSWER: See formula below — know this exactly for the exam

PART 4 — FORMULA DICTIONARY & WORKED SOLUTIONS

Source: DL_FormulaGuide_WorkedSolutions.docx — Each formula broken down symbol-by-symbol with plain-English explanations. Part 4 = Worked Solutions with full step-by-step answers.

PART 1 — COMPLETE FORMULA DICTIONARY

Every important formula from the course, explained for beginners. Formulas are grouped by topic. Read each entry top-to-bottom: Formula → Symbols → Intuition → Example.

TOPIC A: N-Gram Language Models & Smoothing

Formula A1 — N-Gram Probability

| N-gram Conditional Probability |
|
| $P(w_n | w_1, w_2, \dots, w_{n-1}) = C(w_1, w_2, \dots, w_n) / C(w_1, w_2, \dots, w_{n-1})$ |

Symbol Meaning Plain English Explanation

$P(w_n \dots)$	Probability of word w_n given previous $n-1$ words	What is the chance of seeing this word after those earlier words?
$C(w_1 \dots w_n)$	Count of the n -word sequence in training corpus	How many times did these exact n words appear together in training?
$C(w_1 \dots w_{n-1})$	Count of the $(n-1)$ -word prefix in training corpus	How many times did the first $n-1$ words appear together?

► **EXAM TIP:** *if you saw 'the cat sat' 10 times and 'the cat' 50 times, the probability of 'sat' after 'the cat' = $10/50 = 0.2$. You are simply dividing how often the full phrase occurred by how often the context occurred.*

| Mini Example (Bigram): |
|
| Corpus: 'I love deep learning. I love coding.' |
|
| $C('I \text{ love}') = 2, C('I') = 2$ |
|
| $P('love' | 'I') = C('I \text{ love}') / C('I') = 2 / 2 = 1.0$ |
|
| This means 'love' always follows 'I' in our tiny corpus (probability = 1). |

Formula A2 — Laplace (Add-1) Smoothing

| Laplace Smoothed Bigram Probability |
|
| $P_{\text{smooth}}(w_n | w_{n-1}) = [C(w_{n-1}, w_n) + 1] / [C(w_{n-1}) + V]$ |

Symbol Meaning Plain English Explanation

$C(w_{n-1}, w_n)$	Raw count of bigram (w_{n-1}, w_n) in corpus	How many times did this exact word pair appear?
$C(w_{n-1})$	Count of the context word w_{n-1}	How many times did the first word appear alone?

V Vocabulary size (total unique words) How many different words are in our dictionary?
 $+1 / +V$ Smoothing constants We pretend every bigram appeared at least once extra (prevents zero probabilities)

► **EXAM TIP:** we need it: Without smoothing, any unseen bigram gets $P=0/N=0$. One zero anywhere makes the entire sentence probability 0. Laplace adds 1 to every count — seen and unseen — so no word pair is ever 'impossible'.

Formula A3 — Perplexity

| Perplexity of a Language Model on Test Corpus W |
 |
 | $PP(W) = P(w_1, w_2, \dots, w_N)^{-1/N} = (1 / P(w_1 w_2 \dots w_N))^{1/N}$ |

Symbol Meaning Plain English Explanation

PP(W)	Perplexity score (lower = better model) of how 'surprised' the model is by the test text	A measure
$P(w_1 \dots w_N)$	Probability of entire test corpus under the model does the model think this test text is?	How likely

N Total number of tokens in test corpus Length of the text we are evaluating

$^{-1/N}$	Normalise and invert by length so short/long texts are comparable; invert so lower surprise = lower perplexity	Normalise
-----------	--	-----------

► **EXAM TIP:** Perplexity is the 'average branching factor': how many equally-likely words does the model consider at each step? $PP=10$ means: on average, the model was choosing between 10 options. $PP=2$ means near-perfect prediction. A random model on a 10,000-word vocabulary has $PP=10,000$.

TOPIC B: RNN / LSTM / GRU Equations

Formula B1 — Vanilla RNN Hidden State

| Recurrent Hidden State Update |
 |
 | $h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$ |

Symbol Meaning Plain English Explanation

h_t Hidden state at current timestep t The model's 'memory' after seeing word/token at position t

h_{t-1} Hidden state from previous timestep Memory from everything seen so far up to t-1

x_t Input at timestep t (e.g., word embedding) The current word being processed, as a vector of numbers

W_h, W_x Learnable weight matrices The model's learned parameters — trained by gradient descent

b Bias vector A learned offset added to shift the output

$\tanh(\cdot)$	Hyperbolic tangent activation function range [-1, 1]; helps the network learn non-linear patterns	Squishes output to
----------------	--	--------------------

► **EXAM TIP:** Think of h_t as a 'running summary'. At each new word, you mix the new word ($W_x \cdot x_t$) with your current summary ($W_h \cdot h_{t-1}$), then squish it to [-1,1] with tanh. The problem: for long sequences, the earlier words get 'forgotten' — this is the vanishing gradient problem.

Formula B2 — LSTM: All Four Equations

| LSTM Complete Forward Pass (at timestep t)

|
 |
 |

| $f_t = \text{sigmoid}(W_f \cdot [h_{t-1}, x_t] + b_f) \leftarrow$ Forget Gate $i_t = \text{sigmoid}(W_i \cdot [h_{t-1}, x_t] + b_i) \leftarrow$ Input Gate $g_t = \tanh(W_g \cdot [h_{t-1}, x_t] + b_g) \leftarrow$ Candidate Values $o_t = \text{sigmoid}(W_o \cdot [h_{t-1}, x_t] + b_o) \leftarrow$ Output Gate
 $C_t = f_t \odot C_{t-1} + i_t \odot g_t \leftarrow$ Cell State Update $h_t = o_t \odot \tanh(C_t) \leftarrow$ Hidden State |

Symbol Meaning Plain English Explanation

$f_t \in [0,1]$	Forget gate output	How much of the
OLD memory (C_{t-1}) to keep? 0=erase, 1=keep all		

$i_t \in [0,1]$ Input gate output How much of the NEW candidate info (g_t) to write in?

$g_t \in [-1,1]$ Candidate cell content What NEW information could be added to memory (before gating)

$o_t \in [0,1]$	Output gate	What portion of
the current memory (C_t) to expose as hidden state		

C_t Cell state (long-term memory) The LSTM's 'RAM' — modified slowly, can remember for 1000+ steps

h_t Hidden state (short-term memory) What the LSTM 'says' at this step; passed to next layer and next timestep

$\odot$ Element-wise (Hadamard) multiplication Multiply corresponding elements of two vectors

$[h_{t-1}, x_t]$ Concatenation of two vectors Stack the previous hidden state and current input into one long vector

| ► Gate Intuition (Reading a novel): FORGET: After a chapter ends, forget the minor character's name ($f_t \approx 0$ for that slot). |

INPUT: A new character appears — store their traits in memory ($i_t \approx 1$, write new info).		
OUTPUT: At a tense moment, recall the villain's motive from memory to generate dialogue.		

Formula B3 — GRU: Two Gates

| GRU Forward Pass

|

|

|

| $r_t = \text{sigmoid}(W_r \cdot [h_{t-1}, x_t]) \leftarrow$ Reset Gate $z_t = \text{sigmoid}(W_z \cdot [h_{t-1}, x_t]) \leftarrow$ Update Gate $\tilde{h}_t = \tanh(W \cdot [r_t \odot h_{t-1}, x_t]) \leftarrow$ New Content $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \leftarrow$ Final Hidden State |

Symbol	Meaning	Plain English Explanation
--------	---------	---------------------------

r_t	Reset gate	How much of past memory to consider when computing new content? (0=ignore past)
z_t	Update gate	How much to UPDATE: blend of old h_{t-1} and new $\tilde{h}_t$. $z=0 \rightarrow$ keep old, $z=1 \rightarrow$ all new

$\tilde{h}_t$ Candidate new hidden state The proposed new hidden state if we fully reset and rewrite

$(1-z_t) \odot h_{t-1}$ Carry-over from past How much of the old state to preserve unchanged

$z_t \odot \tilde{h}_t$ New contribution How much of the new candidate to blend in

TOPIC C: Seq2Seq + Bahdanau Attention

Formula C1 — Bahdanau Attention Score

| Additive Attention Score (Alignment Model) |

|

| $e_{ij} = v_a^T \cdot \tanh(W_a \cdot s_{i-1} + U_a \cdot h_j)$ |

Symbol	Meaning	Plain English Explanation
--------	---------	---------------------------

e_{ij} Raw alignment score (a scalar) How well does the decoder's current need MATCH encoder position j?

s_{i-1} Decoder hidden state at step $i-1$ (the 'query') What the decoder is currently looking for / trying to generate next

h_j Encoder hidden state at position j (the 'key') What source word j 'means' in context — can attend from both directions

W_a, U_a Learnable weight matrices Project decoder & encoder states into a shared space for comparison

$\tanh(\cdot)$	Non-linear activation	Allows
complex matching beyond simple dot products		
$v_a^T \cdot (\cdot)$	Learnable weight vector $\rightarrow$ scalar	Collapses the
alignment vector down to a single score number		

Formula C2 — Attention Weights (softmax)

| Normalise Scores to Weights (must sum to 1) |

|

| $\alpha_{ij} = \exp(e_{ij}) / \sum_k \exp(e_{ik})$ |

Symbol	Meaning	Plain English Explanation
--------	---------	---------------------------

α_{ij}	Attention weight for source position j at decode step i	What fraction of 'attention' does source word j receive right now?
---------------	---	--

$\exp(\cdot)$	Exponential function
Makes all values positive; amplifies differences between scores	
$\sum_k \exp(\dots)$	Sum over all source positions
Normalising constant so all weights sum to 1 (like a probability)	

Formula C3 — Context Vector

| Weighted Sum of Encoder States |

|

| $c_i = \sum_j [\alpha_{ij} \cdot h_j]$ |

Symbol	Meaning	Plain English Explanation
--------	---------	---------------------------

c_i	Context vector for decode step i	A single summary vector encoding 'what to look at' in the source — passed to the decoder
-------	------------------------------------	--

α_{ij}	Attention weight (from C2 above)	How much to weight each source word's contribution
---------------	----------------------------------	--

h_j	Encoder hidden state at position j	The meaning of source word j in its full context
-------	--------------------------------------	--

► **INFO:** Big Picture Intuition: Imagine translating English $\rightarrow$ French. When generating the French word 'manteau' (coat), the decoder FOCUSES (high α) on the English word 'coat' and low α everywhere else. c_i is a weighted blend of all English word representations — heavily weighted towards 'coat'. This is exactly what attention does: dynamically route information from the source.

TOPIC D: Self-Attention & Transformer

Formula D1 — Scaled Dot-Product Attention

| The Core Transformer Attention Formula |

|

| $\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$ |

Symbol	Meaning	Plain English Explanation
--------	---------	---------------------------

Q (Query)	What each token is LOOKING for (matrix $n \times d_k$)	Each
row = one token's 'search query' — what context do I need?		
K (Key)	What each token OFFERS as an identifier (matrix $n \times d_k$)	Each
row = one token's 'label' — what do I represent?		

V (Value)	The actual CONTENT each token contributes (matrix $n \times d_v$)	Each row = one token's payload – what information do I send?
$Q \cdot K^T$	Dot product: similarity between every Q-K pair ($n \times n$)	How closely does each token's query match each other token's key?

$\text{sqrt}(d_k)$ Square root of key dimension
 softmax into saturation

Scaling to prevent giant dot products that push

$\text{softmax}(\cdot)$ Normalise scores to sum-to-1 weights (attention weights) Converts raw similarity scores to a proper probability distribution

$\cdot V$ Weighted sum of value vectors
 retrieves content.

Blend the information from all tokens, weighted by how relevant they are

► **INFO:** One-Sentence Summary: $Q \cdot K^T$ measures similarity, $\text{sqrt}(d_k)$ prevents saturation, softmax normalises, $\cdot V$ retrieves content.

Formula D2 — Multi-Head Attention

| Multi-Head Attention |
 |
 | $\text{head}_i = \text{Attention}(Q \cdot W_{Q_i}, K \cdot W_{K_i}, V \cdot W_{V_i})$ MultiHead = Concat(head_1, head_2, ..., head_h) $\cdot W_O$ |

Symbol Meaning Plain English Explanation

head_i Single attention head (one perspective) One 'copy' of attention, each attending to a different kind of relationship

$W_{Q_i}, W_{K_i}, W_{V_i}$ Learnable projection matrices for head i Each head PROJECTS Q, K, V into a different subspace to capture different relationships

h	Number of heads (e.g., $h=8$ in BERT-base)	How many parallel perspectives to learn simultaneously
-----	--	--

$\text{Concat}(\cdot)$ Concatenate all head outputs Stack all head outputs side by side into one long vector

W_O Output projection matrix Mix the concatenated heads back to d_{model} dimension

► **EXAM TIP:** multiple heads?: One head might learn 'subject-verb agreement'. Another might learn 'pronoun-antecedent'. A third might learn 'nearby syntax'. All running in parallel — the model gets 8 different views of the sentence simultaneously, then combines them.

Formula D3 — Position-wise Feed-Forward Network (FFN)

| Transformer FFN (applied independently to each position) |

|
 | $\text{FFN}(x) = \max(0, x \cdot W_1 + b_1) \cdot W_2 + b_2$ |

Symbol Meaning Plain English Explanation

x Input vector for one token position (d_{model} -dimensional) The token's current representation after attention

$W_1 \in \mathbb{R}^{(d \times d_{\text{ff}})}$	First linear layer weights ($d_{\text{ff}} = 4 \times d_{\text{model}}$ typically)
Expands the representation to a larger space	

$\max(0, \cdot)$ ReLU activation Sets all negative values to 0 — introduces non-linearity

$W_2 \in \mathbb{R}^{(d_{\text{ff}} \times d)}$	Second linear layer (projects back to d_{model})
Compresses back to original dimension	

b_1, b_2 Bias terms

Learned offsets for each layer

Formula D4 — Sinusoidal Positional Encoding

| Positional Encoding for position pos, dimension $2i$ and $2i+1$ |
 |

| $PE(pos, 2i) = \sin(pos / 10000^{(2i / d_model)})$ | $PE(pos, 2i+1) = \cos(pos / 10000^{(2i / d_model)})$ |

Symbol	Meaning	Plain English Explanation
pos	Position of the token in the sequence (0, 1, 2, ...) Which slot in the sentence? First word=0, second word=1, etc.	
i	Index of the dimension pair (0, 1, 2, ...)	Which pair of dimensions in the embedding are we computing?
d_model	Total embedding dimension (e.g., 512)	Size of the token's full vector representation
$10000^{(2i/d_model)}$	Frequency denominator (different wavelength per dimension) Each dimension oscillates at a different speed — creates unique fingerprints	
$\sin / \cos$	Alternating sine and cosine $\cos$ — ensures uniqueness	Even-indexed dimensions use $\sin$, odd-indexed use $\cos$

► **EXAM TIP:** *sinusoids?: Unique encoding for each position (no two positions have the same PE). Generalises to longer sequences than training (no learned parameters needed). Relative positions can be expressed as linear transformations of PE — helping the model learn relative distance patterns.*

TOPIC E: Loss Functions & Objectives

Formula E1 — Softmax

| Softmax — Convert raw scores (logits) to probabilities |

| $\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$ |

Symbol	Meaning	Plain English Explanation
z_i	Raw score (logit) for class i confidence for class i — can be any real number	The model's un-normalised
$\exp(z_i)$	Exponentiate the score and amplifies the biggest score	Makes all values positive
$\sum_j \exp(z_j)$	Sum of all exponentiated scores all probabilities add up to 1	Normalising constant so

$\text{softmax}(z_i)$ Probability of class i A number between 0 and 1; all classes sum to exactly 1

| Mini Example: |

| |

| $z = [2.0, 1.0, 0.1]$ (raw scores for 3 classes) |

| |

| $\exp(z) = [7.39, 2.72, 1.11]$ |

| |

| $\text{sum} = 7.39 + 2.72 + 1.11 = 11.22$ |

| |

| $\text{softmax} = [7.39/11.22, 2.72/11.22, 1.11/11.22] = [0.659, 0.242, 0.099]$ |

| |

| Class 1 wins with 65.9% probability. |

Formula E2 — Cross-Entropy Loss (Classification)

| Categorical Cross-Entropy Loss (C classes) |

| |

| $L = -\sum_{c=1}^C [y_c \cdot \log(\hat{y}_c)]$ |

Symbol	Meaning	Plain English Explanation
L	Loss value (we MINIMISE this during training) wrong is the model? 0=perfect, higher=worse	How
C	Number of classes	How many different labels are possible? (e.g., 6 for emotion, 2 for sentiment)
y_c	True label for class c (one-hot: 1 if correct, else 0)	The ground truth — only the correct class is 1, all others are 0
$\hat{y}_c$	Model's predicted probability for class c (from softmax)	What probability does the model assign to class c?

```
log(.)    Natural logarithm
Penalises low-confidence correct predictions very harshly (log(0.01) = -4.6)
-SUM      Negative sum
the loss positive (log of a probability < 1 is negative)      Makes
```

Formula E3 — RLHF Regularised Reward Objective

| RLHF Training Objective (what we maximise) |

|

| $J(\theta) = E_{\pi_{\theta}}[r(x, y)] - \lambda \cdot KL(\pi_{\theta} || \pi_{ref})$ |

Symbol	Meaning	Plain English Explanation
$J(\theta)$	Objective to MAXIMISE	The overall score of the policy — we want to make this as high as possible
π_{θ}	Current policy (the LLM being trained)	The model being fine-tuned with RLHF
$r(x, y)$	Reward score for response y to prompt x output? (human preference proxy)	What score does the reward model give to this
$E[r(x, y)]$	Expected reward across all (prompt, response) pairs	Average reward the policy gets — we want to maximise this
$KL(\pi_{\theta} \pi_{ref})$	KL divergence from reference SFT model	How different is our new policy from the original SFT model?

```
lambda      Regularisation coefficient
How much to penalise deviation. High lambda=stay close to reference;
low=focus on reward
```

π_{ref}	Reference policy (frozen SFT model)	The supervised fine-tuned baseline — we don't want to stray too far from it
-------------	-------------------------------------	---

► **EXAM TIP:** the KL term?: Without it, the policy could 'hack' the reward model — e.g., repeat 'I love you' forever if that scores high, but generates useless text. The KL term keeps the policy close to the SFT model, preventing degenerate reward-hacking behaviour.

Formula E4 — Reward Model Training Loss

| Pairwise Preference Loss (Bradley-Terry Model) |

|

| $L_{RM} = -\log(\sigma(r(x, y_w) - r(x, y_l)))$ |

Symbol	Meaning	Plain English Explanation
L_{RM}	Reward model training loss (minimise this)	How wrong is the reward model about human preferences?
y_w	Preferred (WINNER) response from human annotation	The response a human annotator preferred

y_l Rejected (LOSER) response from human annotation The response a human annotator did NOT prefer

$r(x, y)$ Scalar reward score for (prompt, response) pair The reward model's prediction of how good this response is

$r(x, y_w) - r(x, y_l)$ Reward gap between winner and loser We want this to be a large positive number

```
sigma(.)          Sigmoid function: 1/(1+exp(-x))
Maps reward gap to probability: large positive gap → probability close to 1
-log(.)           Negative log — makes big probability = small loss
When reward gap is large and positive, loss → 0 (model is confident and correct)
```

PART 2 — FULLY WORKED QUESTIONS (Step-by-Step)

Worked Question 1 — Bigram Probability & Perplexity

Q1: A training corpus contains these bigram and unigram counts: $C('the\ cat')=4$, $C('cat\ sat')=3$, $C('sat\ on')=2$, $C('on\ the')=5$, $C('the')=10$, $C('cat')=6$, $C('sat')=4$, $C('on')=5$. Vocabulary size $V=20$. (a) Compute the Laplace-smoothed probability of each bigram in the test sentence 'the cat sat on the'. (b) Compute the perplexity of this sentence under the model. [8 marks]

STEP 1 — Recall the Formula

Always write the formula first in the exam before plugging in numbers:

| Formula Used |
| |
| $P_{smooth}(w_2|w_1) = [C(w_1, w_2) + 1] / [C(w_1) + V]$ $PP(W) = P(sentence)^{-1/N}$ |

STEP 2 — Compute Each Smoothed Bigram Probability

The sentence has 5 words → 4 bigrams: (the,cat), (cat,sat), (sat,on), (on,the).

```
Step 1: Bigram P(cat | the): = [C('the cat')+1] / [C('the')+V] =
(4+1)/(10+20) = 5/30 = 0.1667
```

```
Step 2: Bigram P(sat | cat): = [C('cat sat')+1] / [C('cat')+V] =
(3+1)/(6+20) = 4/26 = 0.1538
```

```
Step 3: Bigram P(on | sat): = [C('sat on')+1] / [C('sat')+V] = (2+1)/(4+20)
= 3/24 = 0.1250
```

```
Step 4: Bigram P(the | on): = [C('on the')+1] / [C('on')+V] = (5+1)/(5+20) =
6/25 = 0.2400
```

STEP 3 — Compute Full Sentence Probability

Under the bigram assumption: $P(sentence) = \text{product of all bigram probabilities}$:

| $P(sentence) = P(cat|the) \times P(sat|cat) \times P(on|sat) \times P(the|on)$ |
| |
| $= 0.1667 \times 0.1538 \times 0.1250 \times 0.2400$ |
| |
| $= 0.1667 \times 0.1538 = 0.02564$ |
| |
| $= 0.02564 \times 0.1250 = 0.003205$ |
| |

$$= 0.003205 \times 0.2400 = 0.000769$$

↑ This is a very small number — completely normal for sentence probabilities.

STEP 4 — Compute Perplexity

Sentence has $N = 5$ words. Apply the perplexity formula:

Step 1: State formula: $PP = P(\text{sentence})^{(-1/N)} = (0.000769)^{(-1/5)}$

Step 2: Compute exponent: $-1/N = -1/5 = -0.2 \rightarrow PP = (0.000769)^{(-0.2)}$

Step 3: Take log to simplify: $\log(PP) = -0.2 \times \log(0.000769) = -0.2 \times (-7.17) = 1.434$

Step 4: Convert back: $PP = e^{(1.434)} \approx 4.19$

FINAL ANSWERS:

$$P(\text{cat}|\text{the}) = 5/30 = 0.1667$$

$$P(\text{sat}|\text{cat}) = 4/26 = 0.1538$$

$$P(\text{on}|\text{sat}) = 3/24 = 0.1250$$

$$P(\text{the}|\text{on}) = 6/25 = 0.2400$$

$$P(\text{sentence}) = 0.000769$$

$$\text{Perplexity } PP \approx 4.19$$

Interpretation: $PP \approx 4.19$ means the model was choosing between ~ 4 equally-likely words at each step. Lower perplexity = better model. A random model on $V=20$ words would have $PP=20$.

► Common Exam Mistakes to Avoid: Do NOT forget to add V to the denominator (many students only add 1 to numerator).

Use the number of tokens $N=5$, NOT the number of bigrams $N-1=4$, in the perplexity formula.

If a bigram is missing from training (count=0), smoothing gives $(0+1)/(C(w_1)+V)$ — NOT zero.

Worked Question 2 — LSTM Single Step Forward Pass

Q2: An LSTM with hidden dimension $d_h=2$ has the following (simplified) weight matrices and biases. At timestep t , the input is $x_t=[1, 0]^T$ and previous hidden state $h_{t-1}=[0.5, -0.3]^T$, previous cell state $C_{t-1}=[0.2, 0.4]^T$. For ALL gates, use the same weight: $W=[0.5, 0.2; 0.3, 0.8]$ applied to $[h_{t-1}; x_t]$ concatenated (size $4 \rightarrow 2$), with bias $b=[0.1, 0.1]^T$. Compute h_t and C_t step-by-step. (Use $\text{sigmoid}(x) \approx x$ for $|x| < 0.5$ for simplicity, else use $\sigma(1) \approx 0.731$, $\sigma(-1) \approx 0.269$, $\tanh(0.5) \approx 0.462$, $\tanh(1) \approx 0.762$) [10 marks]

STEP 1 — Recall ALL LSTM Equations

Write out all equations before starting the computation. This earns partial marks even if arithmetic is off.

| LSTM Equations

|

|

|

| INPUT: $[h_{t-1}; x_t] = [0.5, -0.3, 1, 0]^T$ (concat, size 4) $f_t = \sigma(W_f \cdot [h_{t-1}; x_t] + b_f) \leftarrow$ Forget Gate $i_t = \sigma(W_i \cdot [h_{t-1}; x_t] + b_i) \leftarrow$ Input Gate $g_t = \tanh(W_g \cdot [h_{t-1}; x_t] + b_g) \leftarrow$ Candidate $o_t = \sigma(W_o \cdot [h_{t-1}; x_t] + b_o) \leftarrow$ Output Gate $C_t = f_t \odot C_{t-1} + i_t \odot g_t$ $h_t = o_t \odot \tanh(C_t)$ |

STEP 2 — Compute the Pre-activation for All Gates

All gates share the same weight W (simplified). Compute $W \cdot [h_{t-1}; x_t] + b$ once:

| Concatenated input $z: [h_{t-1}; x_t] = [0.5, -0.3, 1.0, 0.0]^T$ |

|

| Note on W : W is 2×4 (maps 4-dim input to 2-dim output). For simplicity, use: |

|

| Row 1 of W : $[0.5, 0.2, 0.3, 0.8]$ |

|

| Row 2 of W : $[0.5, 0.2, 0.3, 0.8]$ (same for this example) |

|

| Dimension 1: $0.5 \times 0.5 + 0.2 \times (-0.3) + 0.3 \times 1.0 + 0.8 \times 0.0 + 0.1$ |

|

| $= 0.25 - 0.06 + 0.30 + 0.00 + 0.10 = 0.59$ |

|

| Dimension 2: $0.5 \times 0.5 + 0.2 \times (-0.3) + 0.3 \times 1.0 + 0.8 \times 0.0 + 0.1 = 0.59$ |

|

| $\rightarrow$ pre-activation $a = [0.59, 0.59]^T$ (same for both dims in this example) |

STEP 3 — Apply Gate Activations

Step 1: Forget gate $f_t = \sigma(a)$: $\sigma(0.59) \approx 0.59$ (using $\sigma(x) \approx x$ for $|x| < 0.5$ gives 0.59 directly; exact=0.643). $\rightarrow f_t = [0.59, 0.59]^T$

|

Step 2: Input gate $i_t = \sigma(a)$: Same pre-activation $\rightarrow i_t = [0.59, 0.59]^T$

|

Step 3: Candidate $g_t = \tanh(a)$: $\tanh(0.59) \approx 0.530$ (between $\tanh(0.5)=0.462$ and $\tanh(1)=0.762$). $\rightarrow g_t \approx [0.53, 0.53]^T$

|

Step 4: Output gate $o_t = \sigma(a)$: $o_t = [0.59, 0.59]^T$

STEP 4 — Update Cell State C_t

Apply the cell state formula: $C_t = f_t \odot C_{t-1} + i_t \odot g_t$

Step 1: Forget part $f_t \odot C_{t-1}$: $[0.59, 0.59] \odot [0.2, 0.4] = [0.59 \times 0.2, 0.59 \times 0.4] = [0.118, 0.236]$

Step 2: Input part $i_t \odot g_t$: $[0.59, 0.59] \odot [0.53, 0.53] = [0.59 \times 0.53, 0.59 \times 0.53] = [0.313, 0.313]$

Step 3: Cell state $C_t = \text{sum}$: $[0.118 + 0.313, 0.236 + 0.313] = [0.431, 0.549]$

STEP 5 — Compute New Hidden State h_t

Apply: $h_t = o_t \odot \tanh(C_t)$

Step 1: $\tanh(C_t)$: $\tanh([0.431, 0.549]) \approx [0.406, 0.502]$ (using $\tanh(0.5) \approx 0.462$ as reference)

Step 2: $h_t = o_t \odot \tanh(C_t)$: $[0.59, 0.59] \odot [0.406, 0.502] = [0.59 \times 0.406, 0.59 \times 0.502] = [0.240, 0.296]$

FINAL ANSWERS:

Cell state: $C_t = [0.431, 0.549]^T$

Hidden state: $h_t = [0.240, 0.296]^T$

Key takeaway: The forget gate (0.59) partially retained the old cell state. The input gate added new candidate values. The output gate decided what fraction of C_t to expose as h_t . In practice, d_h is 128–1024 and W is NOT shared across gates — this is a simplified exam example. |

► **INFO:** Exam Tip: Even if you get the arithmetic wrong, you WILL get marks for: (1) writing correct equations, (2) showing the correct structure of each step, (3) identifying which activation function each gate uses. Never skip the formula-writing step.

PART 3 — ATTENTION PROBLEMS (Fully Solved)

These are the most frequently tested topics. Each problem is solved numerically, step-by-step, exactly as expected in the IIT Bombay exam. Master these and you cover ~20 marks worth of questions.

Attention Problem 1 — Scaled Dot-Product Attention (Numerical)

QA1: Given the following Q, K, V matrices for a 3-token sequence with $d_k = 2$, compute the full attention output matrix step-by-step. $Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$, $K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$, $V = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$. Show: (i) raw scores QK^T , (ii) scaled scores, (iii) softmax weights, (iv) final output. [8 marks]

STEP 1 — State the Formula

Scaled Dot-Product Attention

Attention(Q, K, V) = softmax($Q \cdot K^T / \sqrt{d_k}$) $\cdot V$ Here: $d_k = 2$, $\sqrt{d_k} = \sqrt{2} \approx 1.414$ |

STEP 2 — Compute Raw Scores $S = Q \cdot K^T$

Multiply each query row by each key row (dot product). Q is 3×2 , K^T is $2 \times 3 \rightarrow$ result is 3×3 .

Row 1 of Q = [1,0]. Dot with each row of K:	
$q1 \cdot k1 = [1,0] \cdot [1,0] = 1 \times 1 + 0 \times 0 = 1$	
$q1 \cdot k2 = [1,0] \cdot [0,1] = 1 \times 0 + 0 \times 1 = 0$	
$q1 \cdot k3 = [1,0] \cdot [1,1] = 1 \times 1 + 0 \times 1 = 1 \rightarrow$ Row 1 of S: [1, 0, 1]	
Row 2 of Q = [0,1]. Dot with each row of K:	
$q2 \cdot k1 = [0,1] \cdot [1,0] = 0$	
$q2 \cdot k2 = [0,1] \cdot [0,1] = 1$	
$q2 \cdot k3 = [0,1] \cdot [1,1] = 1 \rightarrow$ Row 2 of S: [0, 1, 1]	
Row 3 of Q = [1,1]. Dot with each row of K:	
$q3 \cdot k1 = [1,1] \cdot [1,0] = 1$	
$q3 \cdot k2 = [1,1] \cdot [0,1] = 1$	
$q3 \cdot k3 = [1,1] \cdot [1,1] = 2 \rightarrow$ Row 3 of S: [1, 1, 2]	
S = QK ^T = [[1, 0, 1],	
[0, 1, 1],	
[1, 1, 2]]	

STEP 3 — Scale by 1/sqrt(d_k)

Divide every element of S by sqrt(2) $\approx$ 1.414:

Step 1: Scaled scores S_scaled = S / 1.414: Row 1: [1/1.414, 0/1.414, 1/1.414] = [0.707, 0.000, 0.707]

Step 2: Row 2: [0/1.414, 1/1.414, 1/1.414] = [0.000, 0.707, 0.707]

Step 3: Row 3: [1/1.414, 1/1.414, 2/1.414] = [0.707, 0.707, 1.414]

STEP 4 — Apply Softmax Row-by-Row

Softmax is applied to EACH ROW independently. For row [a,b,c]: softmax = [e^a, e^b, e^c] / (e^a+e^b+e^c)

ROW 1 — [0.707, 0.000, 0.707]:	
e ^{0.707} = 2.028, e ^{0.000} = 1.000, e ^{0.707} = 2.028	

	Sum = 2.028 + 1.000 + 2.028 = 5.056	
	$\alpha_1 = [2.028/5.056, 1.000/5.056, 2.028/5.056]$	
	= [0.401, 0.198, 0.401]	
	ROW 2 — [0.000, 0.707, 0.707]:	
	$e^{0.000}=1.000, e^{0.707}=2.028, e^{0.707}=2.028$	
	Sum = 1.000 + 2.028 + 2.028 = 5.056	
	$\alpha_2 = [0.198, 0.401, 0.401]$	
	ROW 3 — [0.707, 0.707, 1.414]:	
	$e^{0.707}=2.028, e^{0.707}=2.028, e^{1.414}=4.113$	
	Sum = 2.028 + 2.028 + 4.113 = 8.169	
	$\alpha_3 = [2.028/8.169, 2.028/8.169, 4.113/8.169]$	
	= [0.248, 0.248, 0.504]	

STEP 5 — Compute Output = Attention_Weights · V

Each output row = weighted sum of V rows. V = [[1,2],[3,4],[5,6]]

	Output row 1 = $\alpha_1 \cdot V = 0.401 \cdot v_1 + 0.198 \cdot v_2 + 0.401 \cdot v_3$:	
	Dim 1: $0.401 \times 1 + 0.198 \times 3 + 0.401 \times 5 = 0.401 + 0.594 + 2.005 = 3.000$	
	Dim 2: $0.401 \times 2 + 0.198 \times 4 + 0.401 \times 6 = 0.802 + 0.792 + 2.406 = 4.000$	
	→ Output row 1: [3.00, 4.00]	
	Output row 2 = $\alpha_2 \cdot V = 0.198 \cdot v_1 + 0.401 \cdot v_2 + 0.401 \cdot v_3$:	
	Dim 1: $0.198 \times 1 + 0.401 \times 3 + 0.401 \times 5 = 0.198 + 1.203 + 2.005 = 3.406$	
	Dim 2: $0.198 \times 2 + 0.401 \times 4 + 0.401 \times 6 = 0.396 + 1.604 + 2.406 = 4.406$	
	→ Output row 2: [3.41, 4.41]	
	Output row 3 = $\alpha_3 \cdot V = 0.248 \cdot v_1 + 0.248 \cdot v_2 + 0.504 \cdot v_3$:	

| Dim 1: $0.248 \times 1 + 0.248 \times 3 + 0.504 \times 5 = 0.248 + 0.744 + 2.520 = 3.512$ |

|

| Dim 2: $0.248 \times 2 + 0.248 \times 4 + 0.504 \times 6 = 0.496 + 0.992 + 3.024 = 4.512$ |

|

| $\rightarrow$ Output row 3: [3.51, 4.51] |

| FINAL ANSWER — Attention Output Matrix:

|

|

|

| Attention(Q,K,V) = [[3.00, 4.00],

|

|

|

| [3.41, 4.41],

|

|

|

| [3.51, 4.51]]

|

|

|

| Interpretation: Token 3 (query=[1,1]) attended MOST to itself ($\alpha=0.504$) since $q_3 \cdot k_3=2$ was the highest score. Its output [3.51, 4.51] is closest to $v_3=[5,6]$. Token 1 split attention equally between positions 1 and 3 (both scored 0.707 after scaling). |

► **INFO:** Exam Shortcut: If you forget e^x values: $e^0=1$, $e^{0.5} \approx 1.65$, $e^{0.7} \approx 2.01$, $e^1 \approx 2.72$, $e^{1.4} \approx 4.06$. Write these at the top of your answer sheet. Softmax errors are forgiven if formula is correct.

Attention Problem 2 — Bahdanau Additive Attention (Numerical)

QA2: A Seq2Seq model translates 'I love AI'. Encoder produces 3 hidden states: $h_1=[1,0]$, $h_2=[0,1]$, $h_3=[1,1]$. At decode step $i=1$, the decoder's previous hidden state is $s_0=[1,0]$. Use $W_a=[[1,0],[0,1]]$, $U_a=[[1,0],[0,1]]$, $v_a=[1,1]^T$ (identity matrices for simplicity). Compute: (a) alignment scores $e_{\{1j\}}$ for $j=1,2,3$, (b) attention weights $\alpha_{\{1j\}}$, (c) context vector c_1 . [8 marks]

STEP 1 — State the Bahdanau Formula

| Bahdanau Attention

|

|

| $e_{\{ij\}} = v_a^T \cdot \tanh(W_a \cdot s_{i-1} + U_a \cdot h_j)$ $\alpha_{\{ij\}} = \exp(e_{\{ij\}}) / \sum_k \exp(e_{\{ik\}})$ $c_i = \sum_j \alpha_{\{ij\}} \cdot h_j$ |

STEP 2 — Compute $W_a \cdot s_0$ (decoder projection)

This is computed ONCE and reused for all source positions j .

Step 1: $W_a \cdot s_0 = I \cdot [1,0]^T = [1 \times 1 + 0 \times 0, 0 \times 1 + 1 \times 0]^T = [1, 0]^T$ (identity matrix: output = input)

STEP 3 — Compute Alignment Scores $e_{\{1j\}}$ for Each Source Position

| FOR $j=1$ ($h_1=[1,0]$, represents 'I'): |

|

|

| $U_a \cdot h_1 = I \cdot [1,0]^T = [1,0]^T$ |


```

|                                     |
|  $W_a \cdot s_0 + U_a \cdot h_1 = [1,0] + [1,0] = [2, 0]$  |
|                                     |
|  $\tanh([2,0]) = [\tanh(2), \tanh(0)] = [0.964, 0.000]$  |
|                                     |
|  $e_{\{11\}} = v_a^T \cdot \tanh = [1,1] \cdot [0.964, 0.000] = 0.964 + 0.000 = 0.964$  |
|                                     |
| FOR j=2 (h2=[0,1], represents 'love'): |
|                                     |
|  $U_a \cdot h_2 = [0,1]^T$  |
|                                     |
|  $W_a \cdot s_0 + U_a \cdot h_2 = [1,0] + [0,1] = [1, 1]$  |
|                                     |
|  $\tanh([1,1]) = [\tanh(1), \tanh(1)] = [0.762, 0.762]$  |
|                                     |
|  $e_{\{12\}} = [1,1] \cdot [0.762, 0.762] = 0.762 + 0.762 = 1.524$  |
|                                     |
| FOR j=3 (h3=[1,1], represents 'AI'): |
|                                     |
|  $U_a \cdot h_3 = [1,1]^T$  |
|                                     |
|  $W_a \cdot s_0 + U_a \cdot h_3 = [1,0] + [1,1] = [2, 1]$  |
|                                     |
|  $\tanh([2,1]) = [\tanh(2), \tanh(1)] = [0.964, 0.762]$  |
|                                     |
|  $e_{\{13\}} = [1,1] \cdot [0.964, 0.762] = 0.964 + 0.762 = 1.726$  |

```

STEP 4 — Softmax to Get Attention Weights $\alpha_{\{1j\}}$

```

Step 1: Exponentiate scores:  $e^{0.964} \approx 2.623$ ,  $e^{1.524} \approx 4.590$ ,  $e^{1.726} \approx 5.619$ 

```

```

Step 2: Sum of exponentials:  $Z = 2.623 + 4.590 + 5.619 = 12.832$ 

```

```

Step 3: Attention weights:  $\alpha_{\{11\}} = 2.623/12.832 = 0.204$  (word 'I' gets 20.4% attention)

```

```

Step 4:  $\alpha_{\{12\}} = 4.590/12.832 = 0.358$  (word 'love' gets 35.8% attention)

```

```

Step 5:  $\alpha_{\{13\}} = 5.619/12.832 = 0.438$  (word 'AI' gets 43.8% attention)

```

✓Sanity check: $0.204 + 0.358 + 0.438 = 1.000$ (must sum to 1)

STEP 5 — Compute Context Vector $c_1 = \sum_j \alpha_{\{1j\}} \cdot h_j$

```

|  $c_1 = \alpha_{\{11\}} \cdot h_1 + \alpha_{\{12\}} \cdot h_2 + \alpha_{\{13\}} \cdot h_3$  |
|                                     |
|  $= 0.204 \cdot [1,0] + 0.358 \cdot [0,1] + 0.438 \cdot [1,1]$  |

```

| |
| Dim 1: $0.204 \times 1 + 0.358 \times 0 + 0.438 \times 1 = 0.204 + 0.000 + 0.438 = 0.642$ |

| |
| Dim 2: $0.204 \times 0 + 0.358 \times 1 + 0.438 \times 1 = 0.000 + 0.358 + 0.438 = 0.796$ |

| |
| $c_1 = [0.642, 0.796]^T$ |

| FINAL ANSWERS:

| |
| |
| |
| Alignment scores: $e_{11}=0.964$, $e_{12}=1.524$, $e_{13}=1.726$

| |
| |
| |
| Attention weights: $\alpha_{11}=0.204$, $\alpha_{12}=0.358$, $\alpha_{13}=0.438$

| |
| |
| |
| Context vector: $c_1 = [0.642, 0.796]^T$

| |
| |
| Interpretation: The decoder's first step (generating the first French word) focused most on 'Al' (43.8%), then 'love' (35.8%), and least on 'I' (20.4%). The context vector c_1 is a blend of all three encoder states, dominated by h_3 and h_2 . This context vector is concatenated with s_0 and fed to the decoder output layer to generate the first target word. |

Attention Problem 3 — Multi-Head Attention: Why & How

QA3: Consider the sentence: 'The bank by the river was steep.' (a) Explain why a SINGLE attention head may fail to capture all relevant relationships in this sentence. (b) Show concretely how two different attention heads might specialize for different relationships. (c) Write the MultiHead formula and describe how the outputs are combined. [6 marks]

Part (a) — Limitation of a Single Head

The softmax bottleneck problem:

A single attention head computes ONE attention distribution over ALL positions. Because softmax must produce a single probability distribution, the head tends to focus sharply on ONE dominant relationship — it cannot easily attend to multiple different types of relationships simultaneously.

| Example — 'The bank by the river was steep.'

| |
| |
| For the token 'bank', a single head might:

| |
| |
| • Strongly attend to 'river' (detecting geographic context) — gets 60% weight

| |
| |

| • Weakly attend to 'steep' (property) — gets 25% weight

| • Almost ignore 'The' (article agreement) — gets 5% weight

| Problem: it CANNOT simultaneously also focus on subject-verb agreement ('bank...was') because the attention budget is already spent. One head = one perspective only. |

Part (b) — How Two Heads Specialise

HEAD 1 — Syntactic Role

HEAD 2 — Semantic

Context

For token 'bank': High attention on subject 'bank' → verb 'was' (subject-verb). Low attention on noun phrases. Captures grammar structure. For token 'bank': High attention on 'river', 'steep' (geographic sense). Disambiguates 'bank'=riverbank (not financial). Captures word sense.

Part (c) — Multi-Head Formula and Combination

| Multi-Head Attention

| head_i = Attention(Q·W^Q_i, K·W^K_i, V·W^V_i) for i=1,...,h MultiHead(Q,K,V) = Concat(head₁, ..., head_h) · W_O |

Step 1: Each head gets its own projections: W^Q_i, W^K_i, W^V_i ∈ R^(d_{model} × d_k) where d_k = d_{model}/h. With d_{model}=512 and h=8 heads: d_k = 512/8 = 64. Each head works in 64-dim space.

Step 2: Run attention independently: Each head_i = softmax(Q_i · K_i^T / √64) · V_i → gives a 64-dim output per position.

Step 3: Concatenate all heads: Concat(head₁, ..., head₈) → 8×64 = 512-dim vector per position (back to d_{model}).

Step 4: Output projection W_O: A learnable d_{model}×d_{model} matrix that mixes information across all heads, allowing the model to learn which combinations of head outputs are most useful.

| KEY TAKEAWAYS FOR EXAM:

| • Single head → one type of relationship. Multi-head → h parallel perspectives. |

| • Each head uses DIFFERENT learned projection matrices W^Q_i, W^K_i, W^V_i. |

| • d_k per head = d_{model} / h (total computation stays similar to one full head). |

| • Final step: Concat → W_O → d_{model} output. W_O mixes head outputs meaningfully. |

| • In BERT-base: d_{model}=768, h=12, d_k=64. In BERT-large: d_{model}=1024, h=16, d_k=64. |

Attention Problem 4 — Causal Masking in Decoder Self-Attention

QA4: For a decoder processing a 4-token sequence [A, B, C, D] during training, (a) draw the 4×4 mask matrix M that enforces causality in self-attention, (b) show how masking is applied to the raw attention scores matrix S, and (c) explain WHY this masking is essential. [4 marks]

Part (a) — The Causal Mask Matrix

The mask M has 1 where attention is ALLOWED and $-\infty$ (or $-1e9$) where attention is BLOCKED:

A	B	C	D
0	$-\infty$	$-\infty$	$-\infty$
0	0	$-\infty$	$-\infty$
0	0	0	$-\infty$
0	0	0	0

Green 0 = allowed (current or past). Red $-\infty$ = blocked (future token). Row i can only attend to columns $j \leq i$.

Part (b) — How Masking is Applied

Step 1: Compute raw scores: $S = Q \cdot K^T / \sqrt{d_k}$ (normal attention scores, 4×4 matrix)	
Step 2: Add the mask: $S_{\text{masked}}[i,j] = S[i,j] + M[i,j]$ where $M[i,j] = 0$ (allowed) or $-1e9$ (blocked)	
Step 3: Apply softmax: $\alpha = \text{softmax}(S_{\text{masked}}, \text{ along each row})$	
Step 4: Effect of $-1e9$: $\text{softmax}(-1e9) = e^{(-1e9)} / Z \approx 0 / Z = 0$. Future tokens get exactly 0 attention weight.	

Part (c) — Why Masking is Essential

During TRAINING, the decoder receives the entire target sequence at once (for efficiency). Without masking, when predicting token C, the model would see C and D — it would cheat by looking ahead. The model needs to learn to predict each token using only past context, which is what happens at INFERENCE time (where future tokens genuinely don't exist yet). Masking enforces this autoregressive property during training.

| EXAM ANSWER TEMPLATE:

Mask $M[i,j] = 0$ if $j \leq i$ (can attend to current and past)		
Mask $M[i,j] = -\text{inf}$ if $j > i$ (cannot attend to future)		
Applied as: $S_{\text{masked}} = S + M$, then $\alpha = \text{softmax}(S_{\text{masked}})$		
Purpose: Preserves autoregressive property — prediction at position i depends ONLY on positions 1..i. Mathematically, $e^{(-\text{inf})} = 0$, so masked positions contribute exactly zero to the weighted sum.		

QUICK FORMULA REFERENCE CARD

Tear-out summary — all formulas in one place for last-minute revision.

Formula	Expression	Key Points
---------	------------	------------

N-gram Prob occurrence counts	$P(w_n w_{n-1}) = C(w_{n-1}, w_n) / C(w_{n-1})$	Ratio of co-
Laplace Smooth add V denominator	$P = (C+1)/(C_{ctx}+V)$	Add 1 numerator,
Perplexity model. N = token count	$PP = P(W)^{-1/N}$	Lower = better
RNN Hidden	$h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$	$\tanh$ squishes to $[-1, 1]$
LSTM Forget to KEEP of old cell	$f_t = \sigma(W_f \cdot [h, x] + b)$	$\sigma \rightarrow [0, 1]$; how much
LSTM Input candidate content	$i_t = \sigma(\cdot), g_t = \tanh(\cdot)$	i_t =gate, g_t =new
LSTM Cell vanishing gradient	$C_t = f_t \odot C_{t-1} + i_t \odot g_t$	Additive path $\rightarrow$ no
LSTM Output is exposed	$h_t = o_t \odot \tanh(C_t)$	o_t controls what
Bahdanau Score feedforward alignment network	$e_{ij} = v_a^T \cdot \tanh(W_a \cdot s + U_a \cdot h_j)$	Additive =
Attn Weights positions	$\alpha_{ij} = \exp(e_{ij}) / \sum \exp(e_{ik})$	Softmax over source
Context Vec encoder states	$c_i = \sum_j \alpha_{ij} \cdot h_j$	Weighted sum of
Scaled Attn prevent saturation	$\text{softmax}(QK^T / \sqrt{d_k}) \cdot V$	Scale by $\sqrt{d_k}$ to
MultiHead subspaces $\rightarrow$ combine with W_O	$\text{Concat}(\text{head}_i) \cdot W_O$	h heads $\rightarrow h$

FFN $\max(0, xW_1 + b_1)W_2 + b_2$ Position-wise; ReLU non-linearity

Sin PE odd=cos; no learned params	$\sin(\text{pos}/10000^{(2i/d)})$	Even dims=sin,
Softmax [0,1] and sum to 1	$\exp(z_i) / \sum \exp(z_j)$	All outputs in
Cross-Entropy confidence correct predictions	$L = -\sum y_c \cdot \log(\hat{y}_c)$	Penalises low-

RLHF Objective $E[r(x,y)] - \lambda \cdot \text{KL}(\pi_\theta || \pi_{\text{ref}})$ Maximise reward, penalise deviation from SFT

Reward Model between winner and loser rewards	$L = -\log(\sigma(r(x, y_w) - r(x, y_l)))$	Maximise gap
--	--	--------------

PART 5 — ARCHITECTURE COMPARISON TABLES & CHEAT SHEETS

Source: *DL_Part4_MasterCheatSheet.docx* — Quick-reference comparison tables for RNN vs LSTM vs GRU, BERT vs GPT vs T5 vs BART, prompting strategies, RLHF variants, and one-page topic summaries.

5.1 RNN vs LSTM vs GRU — Side-by-Side

Feature	RNN	LSTM	GRU
Gates	None	3 gates: forget (f), input (i), output (o)	2 gates: reset (r), update (z)
Memory	h_t only	h_t + C_t (cell state)	h_t only (no separate cell)
Parameters	Fewest	Most (~4× RNN)	~3× RNN (middle ground)
Vanishing Gradient	Severe	Mostly solved (cell highway)	Mostly solved
Long-range deps	Poor	Excellent	Good
Training Speed	Fast	Slow	Medium
Best For	Short seqs, simple tasks	Long seqs, complex temporal	When LSTM too slow
Introduced	1986	1997 (Hochreiter)	2014 (Cho et al.)

5.2 BERT vs GPT vs T5 vs BART

Feature	BERT	GPT	T5	BART
Architecture	Encoder only	Decoder only	Encoder-Decoder	Encoder-Decoder
Training Obj.	MLM + NSP	Causal LM (CLM)	Span corruption	Denoising (multi-task)
Attention	Bidirectional	Unidirectional (causal)	Bidir enc, causal dec	Same as T5
Best For	Classification, NER, QA	Generation, completion	Any task (text-to-text)	Summarization, translation
Key Paper	Devlin 2018	Radford 2018	Raffel 2019	Lewis 2019

5.3 Encoder vs Decoder vs Encoder-Decoder

Aspect	Encoder Only (BERT)	Decoder Only (GPT)	Enc-Dec (T5/BART)
Self-Attention	Full bidirectional	Causal (left-to-right)	Enc: bidir Dec: causal
Cross-Attention	None	None	Dec attends to Enc output
Training Signal	Masked positions	Every token (predict next)	Target tokens (seq2seq)
Use Case	Understanding tasks	Generation tasks	Transformation tasks
Inference	One forward pass	Autoregressive	Autoregressive (dec side)

5.4 Prompting Strategies Comparison

Method	Input Format	When to Use	Limitation
Zero-shot	Task description only	General tasks with clear instructions	May fail on novel/complex tasks
Few-shot (ICL)	K examples + task	When examples clarify format/style	Limited by context window
Chain-of-Thought	Examples with reasoning steps	Multi-step reasoning, math, logic	Verbose, can hallucinate
Zero-shot CoT	"Think step by step" suffix	When no examples available	Less reliable than few-shot CoT
Prompt Tuning	Soft prompt prepended (trained)	Production, task-specific	Requires training, not readable
Self-Instruct	Model generates its own data	Bootstrapping instruction data	Quality limited by base model
Evol-Instruct	Evolve/complexify existing data	WizardLM, harder instruction gen	Can drift from original task

5.5 RLHF vs DPO vs RLAIIF

Method	Key Idea	Pros	Cons
RLHF (PPO)	Human prefs → reward model → PPO	Strong alignment, proven (ChatGPT)	Expensive, unstable, slow
DPO	Direct policy optimisation from pref pairs	Simpler, no RL training loop	Less flexible than PPO
RLAIIF	AI (Claude) instead of humans for prefs	Scalable, cheaper labeling	AI biases in reward signal
Self-Play	Model debates itself	No human labels needed	Complex training dynamics

5.6 Transformer Architecture — Key Facts

Attention(Q,K,V) = softmax(QK^T / sqrt(d_k)) * V [Core formula – must know cold]

MultiHead(Q,K,V) = Concat(head_1,...,head_h) * W_O [h heads, each dim = d/h]

FFN(x) = max(0, xW1 + b1)W2 + b2 [Two linear layers, ReLU]

PE(pos,2i) = sin(pos / 10000^(2i/d_model)) [Positional Encoding even dims]

PE(pos,2i+1) = cos(pos / 10000^(2i/d_model)) [Positional Encoding odd dims]

Component	Original Transformer	BERT-Base	GPT-2 Small
Layers (N)	6 encoder + 6 decoder	12 encoder	12 decoder
d_model	512	768	768
Attention Heads (h)	8	12	12
d_ff (FFN)	2048	3072	3072
d_k = d_model/h	64	64	64
Params	~65M	110M	117M

5.7 Tokenization Methods Comparison

Method	Merge Criterion	Prefix for mid-word	Used By	Vocab Size
BPE	Highest frequency pair	Ġ (or space)	GPT, GPT-2, RoBERTa	30k-50k
WordPiece	Highest PMI-like score (log prob)	##	BERT, DistilBERT	30k
Unigram LM	Prune tokens that reduce loss least	No prefix (probabilistic)	T5, ALBERT	32k

5.8 Agentic AI Frameworks Comparison

Framework	Core Idea	Key Feature	Limitation
ReAct	Reason + Act interleaved	Tool calls after reasoning steps	No explicit memory
Reflexion	Verbal RL via self-reflection	Stores reflection in episodic memory	Memory not transferable
ReWOO	Plan ALL tools upfront, then execute	Fewer LLM calls, efficient	Plans can fail if early step fails
Self-Refine	Iterative self-critique + revision	Single model, no tools needed	May loop without converging
HuggingGPT	LLM orchestrates specialist models	Multi-modal, many expert models	Complex, brittle pipelines

PART 6 — QUICK REVISION & PRACTICE QUESTIONS

Source: DL_EndSem_Complete_Notes.docx — True/False, Short Answer questions predicted for the exam. Use these for timed practice.

Section 1: True/False Questions

Question	Answer	Justification
GloVe uses negative sampling like Word2Vec not negative sampling	FALSE	GloVe uses global co-occurrence statistics,
LSTM guarantees no vanishing gradient gradient	FALSE	LSTM reduces (not eliminates) vanishing
Multi-head attention uses the same W_q, W_k, W_v for all heads matrices	FALSE	Each head has its own $W_{q_i}, W_{k_i}, W_{v_i}$
PPO-clip prevents policy updates when ratio is too large preventing large jumps	TRUE	Clip bounds ratio to $[1-\epsilon, 1+\epsilon]$
Teacher forcing is used at inference time uses model outputs	FALSE	Teacher forcing = train time only; inference
BERT uses causal masking in its self-attention attention in encoder	FALSE	BERT uses UNMASKED (bidirectional)
Reflexion fine-tunes model weights using RL weight updates	FALSE	Uses verbal reflection in memory, NO
Perplexity = 2^{entropy} (cross-entropy per word)	TRUE	PP = 2^H by definition
BPE and WordPiece both start with small vocab and expand large and shrinks	TRUE	Both grow; only Unigram LM starts
In WordPiece, an unknown sub-token means only that sub-token is <UNK> <UNK> in WordPiece	FALSE	The WHOLE WORD becomes

Section 2: Short Answer — Predicted Questions

[PREDICTED] Compare BPE, WordPiece, and Unigram LM tokenization. What are the key differences in their merge strategies and test-time behaviour? (5 marks)

► **ANSWER:** BPE: merges most frequent character pair; at test time, applies stored merge rules in sequence; unknown chars $\rightarrow$ <UNK>. WordPiece: merges pair with highest PMI score $\text{freq}(AB)/(\text{freq}(A)\times\text{freq}(B))$; at test time, finds longest vocabulary match left-to-right (no rules stored); unknown sub-token $\rightarrow$ whole word becomes <UNK>. Unigram LM: starts with large vocabulary and REMOVES tokens; uses EM algorithm and Viterbi to find best segmentation; removes tokens that least affect log-likelihood loss. BPE/WordPiece grow vocabulary; Unigram LM shrinks it. BPE $\rightarrow$ GPT; WordPiece $\rightarrow$ BERT; Unigram LM $\rightarrow$ T5 (SentencePiece).

[PREDICTED] Explain the RLHF pipeline from scratch: SFT $\rightarrow$ Reward Model $\rightarrow$ PPO. What is the role of each stage? (5 marks)

► **ANSWER:** (1) SFT: Supervised fine-tuning on curated (instruction, response) pairs. Creates the reference policy π_{ref} . (2) Reward Model: Train RM on human-ranked response pairs (y^+ vs y^-). Loss = $-\log \sigma(r(x, y^+) - r(x, y^-))$ [maximize]. RM learns to predict human preference. (3) PPO: Optimize the policy π_{θ} to maximize: $E[r(x, y) - \beta \cdot \text{KL}(\pi_{\theta} || \pi_{\text{ref}})]$. Reward = high RM score. KL penalty prevents reward hacking. PPO-clip bounds importance ratio to $[1-\epsilon, 1+\epsilon]$ for stable updates.

[PREDICTED] A sequence 'I went to the bank to deposit' — how does BERT represent 'bank' differently than GloVe? (3 marks)

► **ANSWER:** GloVe: 'bank' has ONE fixed vector regardless of context — the average of all contexts (financial + river bank) → confused representation. BERT: processes full sentence through 12+ transformer layers; 'bank' embedding at the output depends on ALL other tokens via bidirectional self-attention. 'deposit' activates financial-context pathways → bank embedding shifts toward financial meaning. This is CONTEXTUAL representation — same word, different vector based on context.

[PREDICTED] Describe the image captioning architecture. What is teacher forcing and why is it used? (4 marks)

► **ANSWER:** Architecture: Encoder (frozen ResNet-50) extracts image features → linear projection to embed_size. Decoder (LSTM, hidden=512) generates captions word-by-word conditioned on: (1) image feature vector and (2) previous word. Teacher forcing: during training, LSTM receives GROUND TRUTH previous word at each step (not its own prediction). Benefits: stable gradients, fast convergence. Problem: exposure bias — at inference, model uses its own outputs which it was never trained on, causing compounding errors.

[PREDICTED] What is POSIX and what are its four evaluation criteria? Why is accuracy alone insufficient? (4 marks)

► **ANSWER:** POSIX (Prompt Sensitivity Index) measures how sensitive a model is to intent-preserving prompt variations. Four criteria: (1) Response Diversity: number of unique responses across variations — more unique = more sensitive. (2) Distribution Entropy: entropy of response distribution — higher = more sensitive. (3) Semantic Coherence: cosine similarity of response embeddings — lower = more sensitive. (4) Confidence Variance: variance in log-likelihood — higher = more sensitive. Accuracy is insufficient because a model can be accurate on one phrasing but wrong on a paraphrase — such a model is unreliable in production where users phrase questions differently.

QUICK REVISION SUMMARIES — LAST-DAY REVIEW (All Topics)

NOTE: Read this page in the final 30 minutes before your exam. Every key formula and exam trap is summarized here.

A: N-gram & Smoothing

INFO: Bigram: $P(w|v) = c(v,w)/c(v)$. Add-1: +1 num, +|V| den. Good-Turing: $c^* = (c+1)N_{\{c+1\}}/N_c$. KN: continuation probability (unique bigrams completed). Perplexity = $P(W)^{-1/N} = 2^H$; lower=better. Problems: sparsity, fixed window, no generalization.

B: RNN/LSTM/GRU

INFO: RNN: $h_t = \tanh(W_h h_{t-1} + W_x x_t)$. Vanishing gradient → product of small Jacobians. LSTM: $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$, $h_t = o_t \odot \tanh(C_t)$. 3 gates: forget/input/output. GRU: 2 gates (update+reset), no cell state. BiRNN: two directions. Teacher forcing: ground truth at train time.

C: Seq2Seq & Attention

INFO: Seq2Seq: encoder → final hidden → decoder (bottleneck!). Beam search: keep top-k (larger k=higher prob). Attention: $e_{ij} = s_i \cdot h_j$, $\alpha_{ij} = \text{softmax}(e_{ij})$, $a_i = \sum \alpha_{ij} V_j$. RAG-Seq+REALM → single doc. RAG reduces hallucination.

D: Transformer

INFO: Self-attention: Q,K,V from same sequence. Scaled: $QK^T/\sqrt{d_k} \cdot V$. Multi-head: H heads, concat+project. Masking: $-\infty$ to future positions. PE: sin/cos added to embeddings. Residual+LayerNorm at each sublayer. Cross-attention: Q=decoder, K,V=encoder.

E: BERT/GPT/T5

INFO: GloVe: NO negative sampling. BERT: MLM(15% masked)+NSP, bidirectional, [CLS]+[SEP]. GPT: causal LM, decoder-only. T5: text-to-text, span masking. BART: denoising. Base models fail zero-shot → NOT trained on instruction-output format.

F: Prompting & IFT

INFO: Zero-shot→no examples. Few-shot→K examples. CoT→reasoning steps. Prompt sensitive to order/format. Self-Instruct=self-generated. Evol-Instruct=Instruction Evolver. Orca=GPT-4 traces. IFT insufficient alone→need RLHF for safety.

G: RLHF

INFO: Pipeline: SFT→RM→PPO. RM loss: $\max \log \sigma(r(x, y^+) - r(x, y^-))$. RLHF obj: $\max E[r(x, y) - \beta \cdot \text{KL}(\pi_\theta || \pi_{\text{ref}})]$.

State=prompt+tokens. Action=token. Reward=EOS. $A(s, a) = Q(s, a) - V(s)$. PPO-clip: $\text{ratio} \in [1 - \epsilon, 1 + \epsilon]$. Constitutional AI: 16 principles, RLAIIF.

H: Agentic AI

INFO: ReAct: Thought-Action-Observation (external tools). Self-Refine: Output→Feedback→Refine (same LLM).

Reflexion: verbal RL, NO weight updates. ReWOO: Plan ALL tools first, execute parallel (token-efficient).

HuggingGPT: Task Planning→Model Selection→Execute→Respond.

I: Tokenization

INFO: BPE=freq merge, grows vocab, apply merge rules at test time, unknown char→UNK. WordPiece=PMI merge, grows vocab, longest match at test time, unknown subword→WHOLE WORD UNK. ULM=shrinks vocab, EM/Viterbi. GPT=BPE, BERT=WordPiece, T5=SentencePiece(ULM).

J: Advanced Prompting

INFO: CoT: few-shot w/ steps OR zero-shot 'Let's think step by step'. Self-Consistency: N chains + majority vote.

ToT: branch+score+backtrack. GoT: connected graph. POSIX: lower=better; few-shot reduces sensitivity; instruction-tuning NOT guaranteed to reduce sensitivity.

K: Code Walkthroughs

INFO: Attention: $Q @ K.T / \text{Vd}_k \rightarrow \text{softmax} \rightarrow @ V$. PE: sin/cos. LSTM stack: pre-pad, `return_sequences=True` for all but last. Sentiment: SimpleRNN+sigmoid. Image caption: ResNet frozen (`requires_grad=False`), LSTM decoder, teacher forcing, split on image IDs.
